

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ  
ФЕДЕРАЦИИ  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ  
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «МОСКОВСКИЙ  
АВИАЦИОННЫЙ ИНСТИТУТ (НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ  
УНИВЕРСИТЕТ)»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по теме:

РЕАЛИЗАЦИЯ ГЛОБАЛЬНОГО ВТОРИЧНОГО ИНДЕКСА И  
СРАВНИТЕЛЬНЫЙ АНАЛИЗ ЕГО ПРОИЗВОДИТЕЛЬНОСТИ С  
ЛОКАЛЬНЫМИ ИНДЕКСАМИ В РАСПРЕДЕЛЕННОЙ СУБД PICODATA

Научный руководитель,  
Канд. техн. наук, доц.

\_\_\_\_\_  
подпись, дата

Романенков А.М.

Исполнитель ВКР,  
Студент группы  
М8О-403Б-22

\_\_\_\_\_  
подпись, дата

Клименко В.М.

Москва 2026

## РЕФЕРАТ

Отчёт 90 с., 13 рис., 7 табл., 22 ист.

РАСПРЕДЕЛЕННЫЕ СУБД, БАЗЫ ДАННЫХ, ВТОРИЧНЫЙ ИНДЕКС, ГЛОБАЛЬНЫЙ ВТОРИЧНЫЙ ИНДЕКС, ЛОКАЛЬНЫЙ ВТОРИЧНЫЙ ИНДЕКС, МАТЕМАТИЧЕСКОЕ МОДЕЛИРОВАНИЕ, СИСТЕМЫ МАССОВОГО ОБСЛУЖИВАНИЯ, ТЕОРИЯ МАССОВОГО ОБСЛУЖИВАНИЯ

Объектом исследования является индексирование данных в распределенных системах управления базами данных (СУБД). Предметом исследования выступает глобальный вторичный индекс (ГВИ) в распределенной СУБД Picodata.

Цель работы — составить математическую модель, по которой можно определить более подходящий тип индекса (локальный или глобальный) в кластере с определенными параметрами.

В работе проводится анализ архитектуры Picodata, описание алгоритмов поиска и обновления в локальных и глобальных вторичных индексах при помощи теории массового обслуживания. Реализован плагин ГВИ к СУБД Picodata и разработано программное обеспечение, позволяющее оценить точность модели с помощью натурных эксперименты.

Основным результатом работы является математическая модель ЛВИ и ГВИ. Новизна работы заключается в моделировании индексных структур в распределенной СУБД с использованием теории массового обслуживания.

Область применения результатов — высоконагруженные распределенные системы, в которых необходим примерный расчет затрат и показателей эффективности без натурных экспериментов.

При помощи методичных материалов можно рассчитать стоимость работы СУБД при использовании различных индексных структур.

Математическая модель не без недостатков — ее можно дополнить вероятностной моделью внутренних структур данных СУБД и особенностями репликации Picodata.

## СОДЕРЖАНИЕ

|                                                                                                |    |
|------------------------------------------------------------------------------------------------|----|
| Термины и определения .....                                                                    | 6  |
| Перечень сокращений и обозначений .....                                                        | 7  |
| Введение .....                                                                                 | 8  |
| 1 Вербальное описание области, направления работы, места и роли<br>продукта, план работы ..... | 9  |
| 1.1 Распределенные системы управления базами данных .....                                      | 9  |
| 1.2 Системы массового обслуживания .....                                                       | 9  |
| 1.3 Проблема поиска .....                                                                      | 10 |
| 1.4 Решение проблемы .....                                                                     | 10 |
| 1.5 Место и роль продукта ВКР .....                                                            | 11 |
| 2 Описание предметной области .....                                                            | 12 |
| 2.1 Верхнеуровневое описание СУБД Picodata .....                                               | 12 |
| 2.1.1 Сегментирование .....                                                                    | 13 |
| 2.2 Диаграмма «сущность-связь» .....                                                           | 14 |
| 2.2.1 Описание условных обозначений .....                                                      | 15 |
| 2.2.2 Описание сущностей .....                                                                 | 15 |
| 2.2.3 Описание связей .....                                                                    | 16 |
| 3 Математическая модель функционирования СУБД Picodata как СМО . .                             | 18 |
| 3.1 СУБД Picodata как СМО .....                                                                | 18 |
| 3.1.1 Показатели эффективности .....                                                           | 20 |
| 3.2 Общие положения модели .....                                                               | 21 |
| 3.3 Поиск в локальном индексе .....                                                            | 23 |
| 3.3.1 Расчет показателей эффективности .....                                                   | 32 |
| 3.4 Поиск в глобальном индексе .....                                                           | 34 |
| 3.4.1 Расчет показателей эффективностей .....                                                  | 43 |
| 3.5 Обновление в локальном индексе .....                                                       | 46 |

|       |                                                             |    |
|-------|-------------------------------------------------------------|----|
| 3.6   | Расчет параметров эффективности .....                       | 50 |
| 3.7   | Обновление в глобальном индексе .....                       | 52 |
| 3.8   | Расчет параметров эффективности .....                       | 57 |
| 3.9   | Результат расчетов и сравнение .....                        | 59 |
| 3.9.1 | Качественное сравнение .....                                | 59 |
| 3.9.2 | Количественное сравнение .....                              | 60 |
| 3.9.3 | Выводы и область применения .....                           | 62 |
| 4     | Модель функционирования плагина .....                       | 64 |
| 4.1   | Контекстная диаграмма .....                                 | 64 |
| 4.2   | Диаграмма потоков данных .....                              | 64 |
| 4.2.1 | Функциональная декомпозиция .....                           | 67 |
| 5     | Архитектура плагина .....                                   | 69 |
| 5.1   | Структурное представление .....                             | 69 |
| 5.2   | Диаграммы развертывания .....                               | 71 |
| 5.2.1 | Развертывание плагина на узле .....                         | 71 |
| 5.2.2 | Развертывание плагина на кластере .....                     | 72 |
| 5.2.3 | Представление данных .....                                  | 73 |
| 5.3   | Итог .....                                                  | 74 |
| 6     | Описание способов определения показателей качества ПО ..... | 76 |
| 6.1   | Показатели качества разрабатываемого плагина .....          | 76 |
| 6.2   | Показатели качества ПО по ГОСТ Р ИСО/МЭК 25010 .....        | 77 |
| 6.3   | Способы определения показателей .....                       | 78 |
| 6.4   | Характеристика полученного результата .....                 | 78 |
| 7     | Программа и методики испытаний ПО .....                     | 80 |
| 7.1   | Объект и цели испытаний .....                               | 80 |
| 7.2   | Состав проверяемых требований .....                         | 80 |
| 7.3   | Методы испытаний .....                                      | 81 |

|                                                                          |    |
|--------------------------------------------------------------------------|----|
| 7.3.1 Функциональная корректность .....                                  | 81 |
| 7.3.2 Производительность операций вставки .....                          | 82 |
| 7.3.3 Сетевые переходы при поиске .....                                  | 82 |
| 7.3.4 Критическая кардинальность .....                                   | 83 |
| 7.3.5 Доступность и надежность .....                                     | 83 |
| 7.3.6 Загрузка CPU .....                                                 | 83 |
| 7.3.7 Контроль доступа .....                                             | 83 |
| 7.4 Средства испытаний .....                                             | 84 |
| 7.5 Условия и порядок проведения испытаний .....                         | 84 |
| 7.6 Критерии приемки .....                                               | 85 |
| 7.7 Сравнение полученных результатов с математическими<br>моделями ..... | 85 |
| Заключение .....                                                         | 87 |
| Список использованных источников .....                                   | 89 |

## ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящем отчете о ВКР применяют следующие термины с соответствующими определениями.

Узел — один запущенный процесс базы данных

Мастер — узел, принимающий запросы на запись и чтение

Реплика — узел, принимающий запросы только на чтение

Набор реплик — узлы, хранящие одинаковую часть данных, среди которых один является мастером, а остальные являются репликами, применяющими поток изменений от мастера

Сегментирование — разбиение данных по наборам реплик

Бакет — единица балансировки. Виртуальная сущность, к которой однозначно привязаны данные

Таблица — совокупность связанных данных, хранящихся в структурированном виде

Первичный ключ — поля таблицы, по которым уникально идентифицируются записи

Вторичный ключ — поля таблицы, по которым неуникально идентифицируются записи

Ключ сегментирования — поля таблицы, по которым происходит определение ее бакета, чаще всего является первичным ключом

Сегментированная таблица — таблица, данные которой распределены между набором реплик

Кластер — сеть набора узлов, предоставляющий доступ к единой СУБД

Плагин — единица программного обеспечения, представленная динамической библиотекой, расширяющая функционал узла

Драйвер — программное обеспечение, предоставляющее интерфейс взаимодействия с кластером

## **ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ**

В настоящем отчете о ВКР применяют следующие сокращения и обозначения.

СУБД — система управления базами данных

ПО — программное обеспечение

ОЗУ — оперативное запоминающее устройство

ЦПУ — центральное процессорное устройство

ЛВИ — локальный вторичный индекс — вторичный индекс, сегментирование которого зависит от ключа сегментирования индексируемой таблицы

ГВИ — глобальный вторичный индекс — вторичный индекс, сегментирование которого зависит от вторичного ключа индексируемой таблицы

## ВВЕДЕНИЕ

Современные распределенные системы управления базами данных (СУБД) обеспечивают горизонтальное масштабирование за счет сегментирования данных, однако эта же архитектура порождает трудности при выполнении запросов, не опирающихся на ключ сегментирования.

Поиск по вторичным атрибутам в таких системах сводится к опросу всех сегментов при использовании локальных вторичных индексов (ЛВИ) — индексной структуры, данные которой сегментированы как индексируемая таблица. Это ограничивает пропускную способность и увеличивает время ответа СУБД. Такая структура увеличивает время работы ЦПУ с ростом количества узлов кластера.

В некоторых СУБД есть решение этой проблемы благодаря глобальным вторичным индексам (ГВИ) — индексной структуре, данные которой сегментированы по вторичным ключам, то есть отдельно от индексируемой таблицы.

Исследование целесообразности внедрения ГВИ в СУБД Picodata при помощи построения математической модели на основе теории массового обслуживания существующего механизма индексирования — ЛВИ — и предложенного, а также создание его программной реализации составляют содержание настоящей работы. Также, математическая модель позволит выбрать более подходящий тип индекса с определенными входными данными кластера и сервиса внедрения СУБД.

Научная новизна работы заключается в построении вышеописанной математической модели — на данный момент такой модели не существует. Это приводит к тому, что возможность оценить работоспособность индексных структур в распределенных СУБД отсутствует. Потребность в оценке появляется из того факта, что нагрузочное тестирование больших кластеров очень дорогостоящее и времязатратное.



## **1 Вербальное описание области, направления работы, места и роли продукта, план работы**

Для понимания прикладной области продукта ВКР, нужно рассмотреть две сущности: распределенные СУБД и системы массового обслуживания (СМО).

### **1.1 Распределенные системы управления базами данных**

Единственные задачи СУБД как хранилища — обновлять пользовательские данные и в будущем искать среди них определенные записи. Для быстрого поиска в памяти подавляющего большинства СУБД поддерживается дополнительная структура данных, строимая над таблицей — индекс.

В распределенных СУБД есть два вида сегментирования индексов:

- над ключом сегментирования индексируемой таблицы;
- над вторичными ключами индексируемой таблицы.

В пункте 1.3 будет показано почему это разделение важно в рамках распределенных СУБД.

### **1.2 Системы массового обслуживания**

Электронные СМО сталкиваются с вызовом эффективной обработки потока заявок, поступающих в случайные моменты времени. В ситуации, когда покупка более производительного аппаратного обеспечения (вертикальное масштабирование) одноканальной СМО становится слишком затратным или попросту невозможным (ввиду законов физики и научного прогресса), переход на многоканальность ради увеличения скорости обработки потока заявок — единственный выход.

Распределенная СУБД Picodata является многоканальной СМО. Ее многоканальность проявляется в сегментировании данных по множеству независимых серверов по значению ключа сегментирования.

### 1.3 Проблема поиска

Положим, что базовой задачей поиска в СУБД является задача поиска по значению ключа сегментирования или вторичного ключа. Сегментирование данных в СУБД Picodata позволяет оптимизировать задачу поиска по ключу сегментирования. Если:

- данные равномерно распределены по всем наборам реплик кластера,
- количество данных в искомой таблице равно  $D$ ,
- количество наборов реплик в кластере равно  $N$ ,
- ПО подключения к СУБД (далее — драйвер) знает функцию, которая по значению ключа сегментирования определяет конкретный набор реплик кластера (далее — функция сегментирования), в котором должна храниться запись, то поиск в структуре данных, хранящей таблицу с искомой записью будет происходить среди  $\frac{D}{N}$  записей, в то время, когда нераспределенным СУБД пришлось бы производить его среди полного набора записей  $D$ .

Однако, в СУБД Picodata нет решения задачи оптимизации поиска по вторичным ключам такой же эффективности. Сейчас происходит опрос кластера с поиском в локальных индексах. То есть среди всех данных  $D$ ,  $N$  процессов параллельно ищут запрашиваемую запись — каждый узел совершает поиск среди  $\frac{D}{N}$  записей.

Рассуждение выше наталкивает на мысль о том, что, например, если значение вторичного ключа однозначно определяет запись (что нередкость), то количество совершённой бесполезной работы при подобном подходе увеличивается пропорционально количеству набору реплик кластера. Помимо этого, количество сетевых запросов растёт пропорционально количеству набору реплик кластера, что тоже в высокой степени влияет на быстродействие поиска в СУБД.

### 1.4 Решение проблемы

Исходя из этого, было решено провести исследовательскую работу по изучению работоспособности глобального вторичного индекса в СУБД

Picodata. Идея заключается в сегментировании индексной структуры по значениям вторичного ключа индексируемой таблицы.

План работ следующий:

- 1) изучить внутренние особенности СУБД Picodata, необходимые для реализации ГВИ;
- 2) описать СУБД Picodata с ЛВИ и ГВИ как СМО;
- 3) построить математическую модель для поиска и обновления ЛВИ и ГВИ в СУБД Picodata: предоставить формулы расчета показателей эффективности СМО;
- 4) провести теоретический расчет показателей эффективности СМО;
- 5) разработать ПО, реализующее ГВИ в виде плагина к СУБД Picodata;
- 6) провести эксперименты для снятия показателей эффективности СМО на разработанном ПО при небольшом размере кластера, сравнить полученные результаты с теоретическим расчетом, объяснить расхождения, и сделать выводы — провести границы применимости распределенного индекса;
- 7) составить справочную информацию по использованию различных типов индексов.

### **1.5 Место и роль продукта ВКР**

Основными продуктами ВКР являются математическая модели ЛВИ и ГВИ в СУБД Picodata и ПО, добавляющее функционал ГВИ в СУБД Picodata.

Функциональные возможности плагина:

- построение глобального вторичного индекса;
- обновление глобального вторичного индекса при изменении данных;
- выполнение поисковых запросов с использованием глобального вторичного индекса.

Целевые пользователи плагина:

- администраторы СУБД Picodata;
- разработчики распределенных приложений.

## 2 Описание предметной области

### 2.1 Верхнеуровневое описание СУБД Picodata

Модель функционирования СУБД Picodata представлена текстовым верхнеуровневым описанием архитектуры, некоторых особенностей и внутренних процессов.

Picodata — это распределенная СУБД, основной фокус которой является отказоустойчивое хранение сегментированных данных. Ее распределенность проявляется при соединении через сеть нескольких запущенных процессов (узлов) Picodata: они собираются в кластер — единую СУБД.

Каждый кластер состоит из нескольких наборов реплик, в каждом наборе мастер-узел обрабатывает пользовательские запросы, а остальные являются его копиями для отказоустойчивого хранения. Разделение данных между наборами реплик позволяет примерно равномерно распределять нагрузку по кластеру. Например, при обновлении одной записи, обновляется только часть базы данных, расположенная на одном наборе реплик.

Picodata представляет из себя многопоточное приложение. Главным потоком является `tx_thread` — в нем происходит доступ к данным и запускается код плагинов — программ, расширяющих возможности СУБД.

При использовании резидентного движка хранения `memtx`, обновления таблицы записываются в журнал упреждающей записи, а индексы хранятся в структуре данных  $B^{+*}$ -дерево (смесь вариантов  $B^{+}$ - и  $B^{*}$ - деревьев) в ОЗУ. При построении вторичного индекса, ключом становится конкатенация значения вторичного ключа с первичным, так что записи с совпадающими вторичными ключами идут подряд. В листе дерева хранится указатель на полную запись в таблице.

Среда потока доступа к данным исполнения является асинхронной, то есть используется кооперативная многозадачность, что позволяет максимально эффективно использовать ЦПУ во время ожидания ответа дискового хранилища или сетевого интерфейса.

### 2.1.1 Сегментирование

Особо важно понимать механизм сегментирования в СУБД Picodata. Равномерность распределения данных, а также предотвращение лишних сетевых запросов при изменении количества наборов реплик достигается благодаря виртуальным сущностям — бакетам, которые однозначно идентифицируются натуральным числом. Количество бакетов в кластере задается единожды при запуске первого узла и не изменяется впоследствии.

Среди наборов реплик бакеты поделены поровну, например, если в кластере 3 набора реплик, а бакетов 3000, то бакеты поделены по 1000 штук (например, равными последовательными отрезками), то есть:

- первому набору реплик принадлежат бакеты с номерами от 1 по 1000;
- второму — от 1001 по 2000;
- третьему — от 2001 по 3000.

При создании сегментированной таблицы, задается набор атрибутов, называемый ключом сегментирования — это поля записи, на основе которых вычисляется бакет каждой записи. Номер бакета равен остатку от деления значения хеш-функции ключа сегментирования.

Любые пользовательские запросы, пришедшие на узел, преобразовываются в план исполнения — набор операций, который должен исполнить движок хранения, чтобы ответить на запрос. Если для исполнения запроса должно быть задействовано несколько наборов реплик, то на мастера каждого набора реплик отправляется план исполнения, который исполняется в локальном движке хранения, например, в случае memtx, это обход и обновление B+\*-дерева в ОЗУ.

Более подробное устройство индекса, построенного на B+\*-дереве можно прочитать в [1–4].

При поиске по ключу сегментирования, узел, принявший запрос, определяет в каком наборе реплик должна быть запись по рассчитанному бакету, и отправляет в него план поиска в индексной структуре.

При поиске не по ключу сегментирования, определить набор реплик записи невозможно, поэтому происходит опрос всех наборов реплик на

наличие искомой записи (как раз этот случай и должен быть оптимизирован глобальным вторичным индексом).

Полную документацию СУБД Picodata можно прочитать на портале документации компании [5]. Подробнее про концепты распределенных систем можно прочитать в [6].

## 2.2 Диаграмма «сущность-связь»

Инфологическая схема предметной области представлена в виде диаграммы «сущность-связь» на рисунке 1.

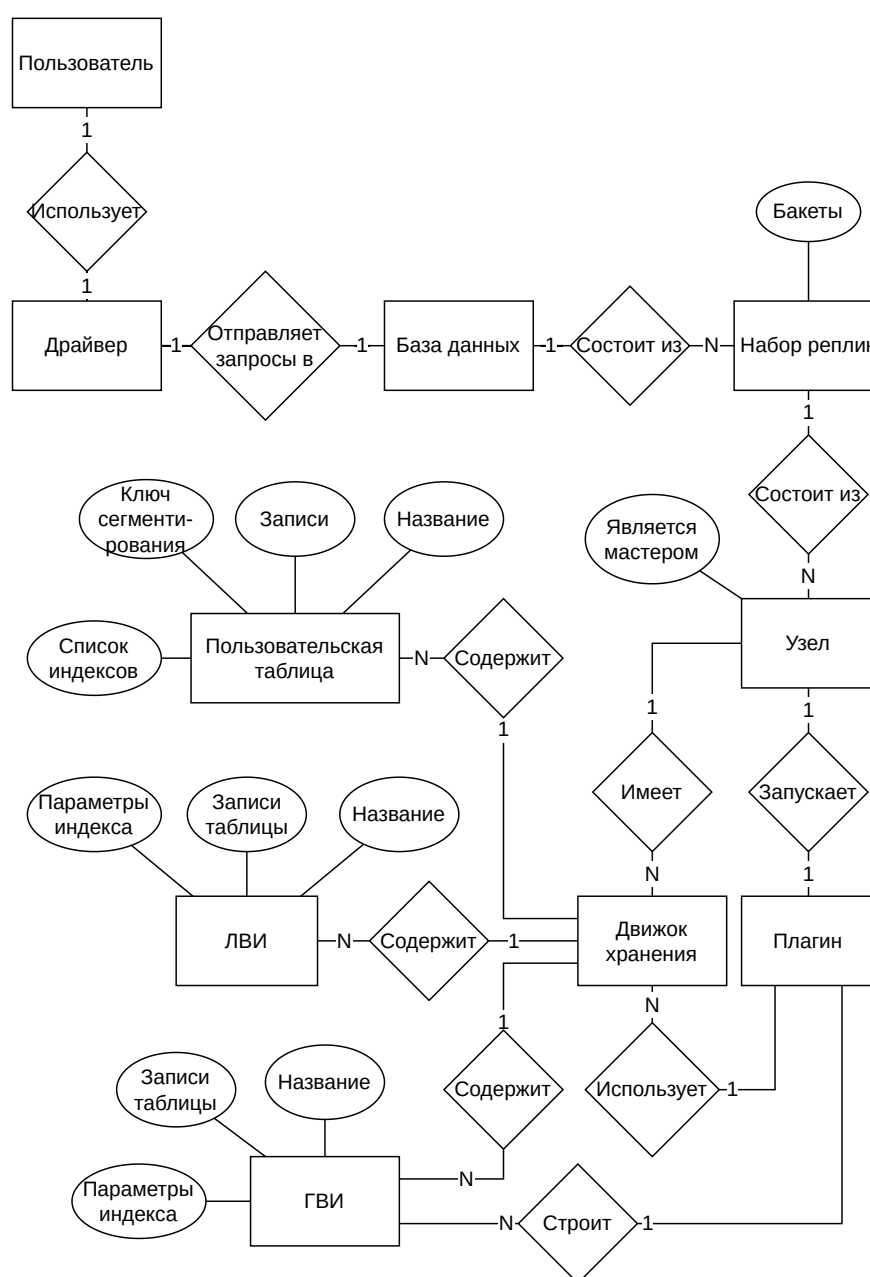


Рисунок 1 — ER-диаграмма предметной области продукта ВКР

### 2.2.1 Описание условных обозначений

Диаграмма выполнена в нотации Чена [7]. Используемые обозначения:

- прямоугольник — сущность;
- ромб — связь;
- овал — атрибут сущности;
- 1:1 — связь «один к одному»;
- 1:N — связь «один ко многим».

Нотация описана в работе Р. Р. Chen «The Entity-Relationship Model — Toward a Unified View of Data» (1976) [7].

### 2.2.2 Описание сущностей

В модели выделены следующие сущности и их атрибуты (если они есть):

- пользователь — лицо, инициирующее запросы к СУБД;
- драйвер — ПО, предоставляющее интерфейс взаимодействия с СУБД;
- база данных — кластер СУБД Picodata;
- набор реплик — набор узлов, хранящих одинаковую часть данных, в котором один является мастером, а остальные являются репликами, принимающими и применяющими поток изменений от мастера.

Атрибуты:

- распределение бакетов — список бакетов, принадлежащих данному набору реплик;
- узел — один запущенный процесс Picodata. Атрибуты:
  - является мастером — булево значение, показывающее роль в наборе реплик: мастер или реплика;
- движок хранения — механизм хранения данных;
- пользовательская таблица — совокупность связанных данных о пользователях сервиса, хранящихся в структурированном виде.

Атрибуты:

- название — идентификатор таблицы;

- первичный ключ — список идентификаторов полей, уникально определяющих запись;
- записи — пользовательские данные;
- список индексов — набор построенных над таблицей индексов;
- ЛВИ — индексная структура, сегментированная как индексируемая таблица. Атрибуты:
  - название — идентификатор индекса;
  - записи таблицы — проиндексированные пользовательские данные;
  - параметры индекса — значения настроек индекса: уникальность значений вторичных ключей;
- плагин — единица программного обеспечения, расширяющая функционал узла;
- ГВИ — индексная структура, сегментированная по индексируемым полям. Атрибуты:
  - название — идентификатор индекса;
  - записи таблицы — проиндексированные пользовательские данные;
  - параметры индекса — значения настроек индекса: уникальность значений вторичных ключей;

### 2.2.3 Описание связей

В модели выделены следующие связи между сущностями:

- пользователь — драйвер: пользователь использует драйвер для отправления запросов к СУБД (1:1);
- драйвер — база данных: драйвер посылает запросы в базу данных (1:1);
- база данных — набор реплик: база данных состоит из наборов реплик (1:N);
- набор реплик — узел: набор реплик состоит из узлов (1:N);
- узел — движок хранения: узел имеет несколько движков хранения (1:N);



- движок хранения — пользовательская таблица: движок хранения содержит множество пользовательских таблиц (1:N);
- движок хранения — ЛВИ: движок хранения содержит множество локальных вторичных индексов (1:N);
- движок хранения — ГВИ: движок хранения содержит множество глобальных вторичных индексов (1:N);
- узел — плагин: узел запускает плагин построения глобальных вторичных индексов (1:1);
- плагин — движок хранения: плагин использует движки хранения для построения глобальных вторичных индексов (1:N);
- плагин — ГВИ: плагин строит множество глобальных вторичных индексов (1:N).

### **3 Математическая модель функционирования СУБД Picodata как СМО**

В данной главе построена математическая модель СУБД Picodata, основываясь на теории массового обслуживания. Рассматриваются локальный вторичный индекс — единственная индексная структура, реализованная в Picodata на данный момент, и новая, предлагаемая структура — глобальный вторичный индекс в контексте обновления данных и поиска по ним.

Это необходимо для математического обоснования создания альтернативного алгоритма индексирования данных в распределенной СУБД Picodata.

В качестве материала для изучения теории массового обслуживания были использованы работы [8–10].

#### **3.1 СУБД Picodata как СМО**

В контексте работы с индексами в СУБД Picodata, удобно разделить каждый узел на две роли — координатора и исполнителя. Такое разделение приведено сугубо для построения математической модели, на деле же все узлы являются одним и тем же приложением с идентичной логикой. Роль узла «зафиксирована» только на время обработки одного запроса и зависит исключительно от выбора пользователем узла, который примет запрос, то есть координатора.

Алгоритм работы координатора представлен следующим циклом:

- 1) принять запрос пользователя;
- 2) составить планы исполнения для мастеров нужных наборов реплик (исполнителей);
- 3) разослать планы исполнителям;
- 4) дождаться результата от исполнителей;
- 5) объединить полученные данные;
- 6) отправить пользователю запрашиваемые данные.

Работа исполнителя происходит в пунктах 3 и 4 работы координатора и ее алгоритм представлен следующим циклом:

- 1) принять план исполнения от координатора;
- 2) исполнить план с помощью движка хранения на своей части данных;
- 3) отправить результат исполнения обратно координатору.

Получается координатор «управляет» всем кластером на время запроса, а исполнитель — только локальными структурами данных. Координатор бьет изначальный запрос на подзадачи, которые решают несколько исполнителей.

В качестве СМО будет описан именно исполнитель, так как работа координатора занимает малое время. Это будет показано в пункте 3.3.

Чтобы избежать излишней сложности модели, постановим, что каждый набор реплик кластера состоит из одного узла. Это не влияет на точность модели, так как реплики являются резервным хранилищем, на котором не исполняются запросы.

В таблице 1 приведена характеристика узла СУБД Picodata как СМО. Положения таблицы верны для обеих индексных структур и являются справедливыми как для поиска, так и для обновления:

Таблица 1 — Характеристика роли координатора узла в СУБД Picodata как СМО

| Элемент СМО        | Элемент СУБД                                                                                        |
|--------------------|-----------------------------------------------------------------------------------------------------|
| Канал обслуживания | tx_thread                                                                                           |
| Поток заявок       | Запросы на поиск или обновление данных сегментированной таблицы                                     |
| Поток ответов      | Отфильтрованные данные таблицы в случае поиска и подтверждение операции в случае обновления таблицы |
| Дисциплина очереди | Очередь бесконечной длины с ограничением по времени ожидания (возможен таймаут заявки)              |

В контексте поиска по индексу, элементарной задачей является нахождение записей с фильтрацией по значению одного вторичного ключа.

В контексте обновления индекса, элементарной задачей является обновление одной записи во вторичном индексе.

Для простоты модели, допустим, что в течение работы количество заявок является случайной величиной следующей закону распределения Пуассона (время между заявками следует экспоненциальному закону распределения) с параметром  $\lambda$ , причем этот поток обладает свойствами:

- стационарности — число заявок не зависит от времени начала работы СМО;
- ординарности — в бесконечно малый промежуток времени шанс получения нескольких заявок стремится к нулю;
- является потоком без последствия — количество заявок в настоящем не зависит от количества заявок в прошлом.

Более подробное описание этих свойств можно найти в [9].

Для расчета задержки сетевых запросов, будем использовать случайную величину с гамма-распределением согласно исследованиям [11,12].

Для расчета времени работы узла, будем ориентироваться на количество операций сравнения ключей при поиске и обновлении в  $B^+$ -дереве, деленное на частоту проведения этих операций ЦПУ и количество операций загрузки данных по указателям, деленное на частоту загрузки блока памяти из запоминающего устройства.

Также, будем учитывать время, потраченное на запись в журнал упреждающей записи, и скажем, что оно константное, так как представляет из себя просто добавление байтов в конец файла, указатель на конец которого известен в момент записи.

Итого, кластер СУБД Picodata моделируется как  $N$  независимых (из-за различных наборов данных) СМО  $M/G/1$  согласно нотации приведенной в [10], где  $N$  — это количество наборов реплик в кластере.

### **3.1.1 Показатели эффективности**

Возьмем следующие показатели эффективности СМО:

- максимальное количество запросов в секунду;
- вероятность необслуживания заявки, то есть вероятность таймаута заявки;

- среднее время обслуживания заявки;
- среднее время ожидания заявки в очереди.

Для построения расчетов показателей эффективности будем использовать следующие параметры:

- количество наборов реплик в кластере  $N$ ;
- объем пользовательских данных  $D$  (количество записей в таблице);
- количество уникальных вторичных ключей (кардинальность множества вторичных ключей)  $K$ ;
- размер одной записи  $r$  (в байтах);
- частота операций сравнения в секунду на ЦПУ  $f_{\text{cpu}}$ ;
- частота операций загрузки и записи данных по указателю из устройства памяти  $f_{\text{mem}}$ ;
- порядок B+\*-дерева  $m$ ;
- время таймаута  $T_{\text{timeout}}$  в секундах;
- скорость сети между пользователем и кластером СУБД  $v_{\text{user}}$  (байтов в секунду);
- скорость сети между узлами  $v_{\text{cluster}}$  (байтов в секунду);
- значения параметров случайных величин:
  - $\lambda_{\text{search}}$  и  $\lambda_{\text{update}}$  — интенсивности потоков заявок (количество заявок в секунду) на поиск и обновление соответственно;
  - $k_{\text{user}}$  и  $\theta_{\text{user}}$  — параметры гамма-распределения  $\Gamma(k_{\text{user}}, \theta_{\text{user}})$  времени сетевой задержки между пользователем и кластером СУБД. Математическое ожидание такого распределения равно  $k_{\text{user}}\theta_{\text{user}}$ , дисперсия равна  $k_{\text{user}}\theta_{\text{user}}^2$ ;
  - $k_{\text{cluster}}$  и  $\theta_{\text{cluster}}$  — параметры гамма-распределения  $\Gamma(k_{\text{cluster}}, \theta_{\text{cluster}})$  времени сетевой задержки между узлами СУБД. Математическое ожидание такого распределения равно  $k_{\text{cluster}}\theta_{\text{cluster}}$ , дисперсия равна  $k_{\text{cluster}}\theta_{\text{cluster}}^2$ .

### 3.2 Общие положения модели

Для обеих индексных структур и обоих видов запросов взаимодействие пользователя на уровне кластера выглядит одинаково: координатор принимает запрос пользователя, по некому алгоритму

собирает данные со всех исполнителей и возвращает их пользователю. Для удобства, расчет времени этого взаимодействия вынесено в лемму 3.2.1.

**Лемма 3.2.1** (о времени ожидания пользователя при запросе в СУБД Picodata)

Время ожидания пользователя исполнения заявки  $T^{\text{user}}$  является случайной величиной и рассчитывается по следующей формуле:

$$T^{\text{user}} = t_{\text{request}}^{\text{user}} + \frac{Q}{v_{\text{user}}} + W^{\text{coordinate}} + t_{\text{response}}^{\text{user}} + \frac{R}{v_{\text{user}}}, \quad (1)$$

где  $t_{\text{request}}^{\text{user}}$  — время задержки передачи запроса от пользователя к СУБД по сети;  $Q$  — размер пользовательского запроса;  $v_{\text{user}}$  — скорость передачи данных между кластером СУБД и пользователем;  $W^{\text{coordinate}}$  — время ожидания обработки запроса координатором;  $t_{\text{response}}^{\text{user}}$  — время задержки передачи результата от СУБД к пользователю по сети;  $R$  — размер результата запроса.

### Доказательство

Взаимодействие пользователя с кластером начинается с отправки запроса по сети.

Задержку сети между пользователем и СУБД, то есть время между отправлением первого байта пользователем и его получением СУБД, моделируется как гамма-распределение, согласно рассуждениям в описании координатора, обозначим ее как  $t_{\text{request}}^{\text{user}} \sim \Gamma(k_{\text{user}}, \theta_{\text{user}})$ .

Передача по сети запроса, весящего  $Q$  байтов со скоростью  $v_{\text{user}}$  байтов в секунду занимает  $Q/v_{\text{user}}$  секунд.

Положим, что время ожидания в очереди и обработки запроса координатором занимает  $W^{\text{coordinate}}$  секунд.

После получения результата в СУБД, он должен быть отправлен обратно пользователю по сети. Задержку сети между СУБД и пользователем моделируем так же — обозначим ее как  $t_{\text{response}}^{\text{user}} \sim \Gamma(k_{\text{user}}, \theta_{\text{user}})$ .

Передача по сети результата, весящего  $R$  байтов со скоростью  $v_{\text{user}}$  байтов в секунду занимает  $R/v_{\text{user}}$  секунд.

При получении результата, взаимодействие пользователя с кластером кончается.

Итого, время ожидания пользователя, которое мы обозначим  $T^{\text{user}}$  является суммой вышеупомянутых величин:

$$T^{\text{user}} = t_{\text{request}}^{\text{user}} + \frac{Q}{v_{\text{user}}} + W^{\text{coordinate}} + t_{\text{response}}^{\text{user}} + \frac{R}{v_{\text{user}}}. \quad (2)$$

Часть суммы — случайная величина, значит и  $T^{\text{user}}$  — случайная величина. ■

В пунктах 3.3, 3.4, 3.5 и 3.7 рассматриваются алгоритмы конкретных индексных структур и запросов. Переменные, специфичные индексной структуре и запросу будут помечены в нижнем индексе. То есть, например, время ожидания исполнения запроса координатора при поиске в ЛВИ будет обозначаться  $W_{\text{search,LSI}}^{\text{coordinate}}$ , а время ожидания пользователем обновления ГВИ будет обозначаться  $T_{\text{update,GSI}}^{\text{user}}$ .

Также, в вышеупомянутых пунктах представлены диаграммы последовательности соответствующих алгоритмов на кластере. Каждая стрелка, переходящая между активностями (activity из UML) объектов (object из UML) представляет из себя один сетевой запрос.

На диаграммах рассмотрены максимально обобщенные топологии кластера, поэтому, блоками с пометкой «цикл» показаны сообщения (messages из UML), которые повторяются. В квадратных скобках внутри этих блоков написаны параметры цикла.

Нотация диаграмм описана в работе [13].

### 3.3 Поиск в локальном индексе

Локальные вторичные индексы устроены следующим образом:

- как сказано в пункте 2.1, данные сегментированных таблиц распределены примерно равномерно среди наборов реплик, а принадлежность записи к набору реплик определяется значением ключа сегментирования;

- часть индекса находится на каждом наборе реплик, причем данные индекса разделены по тому же ключу сегментирования, что и индексируемая таблица.

Алгоритм поиска в кластере с такой индексной структурой выглядит следующим образом:

- 1) координатор принимает пользовательский запрос, составляет план локального поиска;
- 2) координатор отправляет план локального запроса всем исполнителем (мастеру каждого набора реплик);
- 3) на каждом исполнителе происходит поиск записей по значению вторичного ключа в  $B^{+}$ -дереве части индекса;
- 4) каждый исполнитель отправляет найденные записи координатору;
- 5) координатор ждет ответов от всех исполнителей, объединяет их и отправляет найденные записи пользователю.

На рисунке 2 представлена диаграмма последовательности описанного алгоритма. На ней изображены линии жизни (lifelines из UML) следующих объектов:

- клиент — инициатор обращения к СУБД;
- координатор;
- исполнитель  $i$  — исполнитель набора реплик под номером  $i$ .



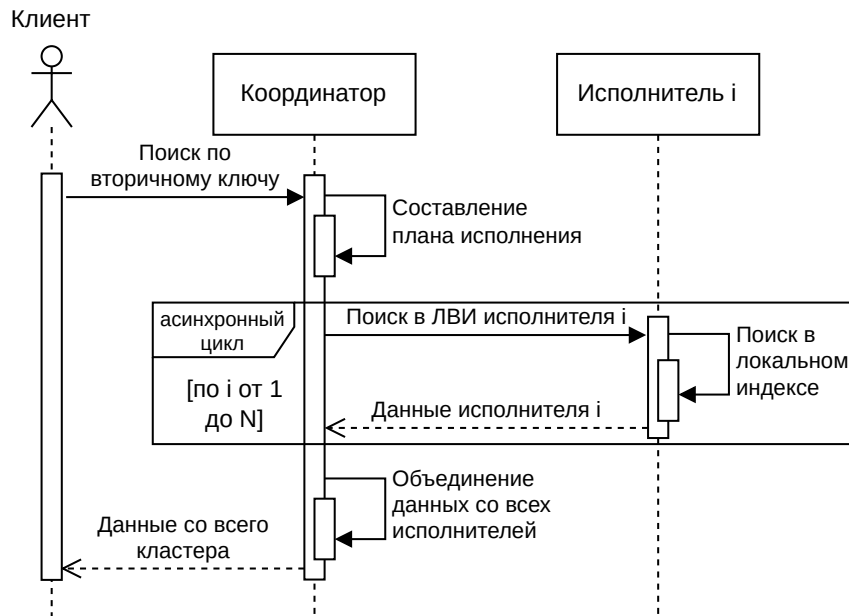


Рисунок 2 — UML Sequence диаграмма поиска в сегментированной таблице в СУБД Picodata с использованием ЛВИ

Исходя из этой диаграммы, можно рассчитать количество сетевых запросов при поиске с использованием ЛВИ ( $NR_{search}^{LSI}$ ). Координатор отправляет план исполнения в каждый набор реплик, из этого выходит  $N$  сетевых запросов, на каждый запрос нужно отправить ответ — умножаем на два; к этому прибавим запрос поиска от клиента к координатору и ответ, итого, получим:

$$NR_{search}^{LSI} = 2N + 2. \quad (3)$$

Таким образом, архитектура ЛВИ приводит к линейной зависимости числа сетевых запросов от количества наборов реплик, что ограничивает горизонтальное масштабирование кластера при поисковых нагрузках.

Данная диаграмма наглядно показывает, что для поиска в кластере с использованием ЛВИ нужно опросить каждый набор реплик на принадлежность записи к сегментированной таблице. Это может оказаться узким горлышком при некоторых ситуациях:

- в кластере много наборов реплик;
- взаимодействие узлов происходит по нестабильной сети;
- кардинальность множества значений индексируемого атрибута стремится к количеству записей в таблице.

Следовательно, возникает необходимость архитектурного решения, позволяющего:

- уменьшить количество сетевых взаимодействий;
- устранить необходимость широковещательного запроса;
- локализовать поиск.

Рассмотрим из чего состоит время ожидания исполнения запроса координатором.

**Теорема 3.3.1** (о времени ожидания исполнения запроса координатором при поиске в ЛВИ)

Время ожидания исполнения запроса координатором  $W_{\text{search,LSI}}^{\text{coordinate}}$  является случайной величиной и рассчитывается по следующей формуле:

$$\begin{aligned} W_{\text{search,LSI}}^{\text{coordinate}} &= W_{\text{search,LSI}}^{\text{execute,q}} + T_{\text{search,LSI}}^{\text{coordinate}} = W_{\text{search,LSI}}^{\text{execute,q}} + \max_{i=1..N} [T_{\text{search,LSI},i}^{\text{coordinate}}] = \\ &= W_{\text{search,LSI}}^{\text{execute,q}} + \max_{i=1..N} \left[ t_{\text{request},i}^{\text{replicaset}} + \frac{P}{v_{\text{cluster}}} + W_{\text{search,LSI}}^{\text{execute}} + t_{\text{response},i}^{\text{replicaset}} + \frac{sr}{v_{\text{cluster}}} \right], \end{aligned} \quad (4)$$

где  $W_{\text{search,LSI}}^{\text{execute,q}}$  — время ожидания запроса в очереди исполнителя;  $T_{\text{search,LSI}}^{\text{coordinate}}$  — время ожидания ответа от всех исполнителей;  $T_{\text{search,LSI},i}^{\text{coordinate}}$  — время ожидания ответа от исполнителя под номером  $i$ ;  $t_{\text{request},i}^{\text{replicaset}}$  и  $t_{\text{response},i}^{\text{replicaset}}$  — времена задержки передачи запроса между координатором и исполнителем под номером  $i$  по сети;  $P$  — размер плана исполнения;  $v_{\text{cluster}}$  — скорость передачи данных между узлами СУБД;  $W_{\text{search,LSI}}^{\text{execute}}$  — время ожидания обработки запроса исполнителем;  $s$  — количество записей с искомым вторичным ключом на наборе реплик;  $r$  — размер одной записи.

### Доказательство

Для начала рассмотрим время, которое проводится исключительно на самом координаторе.

Запрос сначала попадает в очередь исполнителя (так как координатор и исполнитель — это одно и то же приложение, то и очередь у них одна) и находится там  $W_{\text{search,LSI}}^{\text{execute,q}}$  секунд.

После этого происходит составление плана запроса, время которого не учитывается, так как на практике занимает чуть больше 6 микросекунд

на ЦПУ со скоростью 4.46 ГГц, что на 4 порядка меньше чем среднее время пинга домашнего роутера с передачей 56 байтов, подключенного по Wi-Fi — 23 миллисекунды. Если пинг домашнего роутера занимает порядка десяти миллисекунд, тогда можно предположить, что передача сообщений с результатами между узлами будет занимать не меньше времени, особенно если узлы расположены отдаленно географически.

Объединение результатов со всех исполнителей происходит в ОЗУ координатора в векторе, что занимает несколько микросекунд, так что не учитывается в модели.

Итого, на координаторе заявка обрабатывается  $W_{\text{search,LSI}}^{\text{execute,q}}$  секунд, далее он делегирует работу исполнителям. Рассмотрим сколько времени занимает обработка поиска на исполнителях.

Составленный план запроса отправляется каждому исполнителю по сети, которых всего  $N$  штук. Каждое отправление плана сопровождается сетевой задержкой, которая длится  $t_{\text{request},i}^{\text{replicaset}}$ , моделируемая как гамма-распределение  $\Gamma(k_{\text{cluster}}, \theta_{\text{cluster}})$ , аналогично лемме 3.2.1.

Положим, что план запроса весит  $P$  байтов, а скорость передачи данных между узлами кластера равна  $v_{\text{cluster}}$  байтов в секунду. Тогда весь план будет передан через  $P/v_{\text{cluster}}$  секунд.

На исполнителе время ожидания исполнения заявки, то есть время, проведенное в очереди в сумме с временем непосредственной обработки заявки обозначим как  $W_{\text{search,LSI}}^{\text{execute}}$ .

Результат работы каждого исполнителя отправляется координатору отдельным сетевым запросом с задержкой  $t_{\text{request},i}^{\text{replicaset}}$ , моделируемой гамма-распределением  $\Gamma(k_{\text{cluster}}, \theta_{\text{cluster}})$ .

Исходя из предположения, что данные распределены равномерно (в том числе по вторичным ключам) по наборам реплик, то на каждом исполнителе находится  $s$ , равное  $\frac{D}{NK}$  искомых записей.

Положим, что одна запись весит  $r$  байтов, тогда передача данных с одного исполнителя занимает  $\frac{sr}{v_{\text{cluster}}}$  секунд.

Итого, время, которое занимает прием и обработка плана исполнения на одном исполнителе является следующей суммой:

$$t_{\text{request},i}^{\text{replicaset}} + \frac{P}{v_{\text{cluster}}} + W_{\text{search,LSI}}^{\text{execute}} + t_{\text{response},i}^{\text{replicaset}} + \frac{sr}{v_{\text{cluster}}}, \quad (5)$$

которую мы обозначим  $T_{\text{search,LSI},i}^{\text{coordinate}}$ .

Координатор должен дождаться ответа от всех исполнителей, чтобы вернуть объединенный результат пользователю, так что время ожидания исполнителей равно времени ожидания самого долгого исполнителя, что является максимумом среди всех  $T_{\text{search,LSI},i}^{\text{coordinate}}$ .

Итого, время ожидания исполнения запроса координатором  $W_{\text{search,LSI}}^{\text{coordinate}}$  рассчитывается по следующей формуле:

$$\begin{aligned} W_{\text{search,LSI}}^{\text{coordinate}} &= W_{\text{search,LSI}}^{\text{execute,q}} + T_{\text{search,LSI}}^{\text{coordinate}} = W_{\text{search,LSI}}^{\text{execute,q}} + \max_{i=1..N} [T_{\text{search,LSI},i}^{\text{coordinate}}] = \\ &= W_{\text{search,LSI}}^{\text{execute,q}} + \max_{i=1..N} \left[ t_{\text{request},i}^{\text{replicaset}} + \frac{P}{v_{\text{cluster}}} + W_{\text{search,LSI}}^{\text{execute}} + t_{\text{response},i}^{\text{replicaset}} + \frac{sr}{v_{\text{cluster}}} \right]. \end{aligned} \quad (6)$$

В правой части равенства присутствуют случайные величины, так что и левая часть является случайной величиной. ■

Заметим, что каждая заявка на поиск порождает  $N$  заявок исполнителям, причем интенсивность потока исполнителя  $\lambda_{\text{search,LSI}}^{\text{sub}}$  так же равна  $\lambda_{\text{search,LSI}}$ . Интенсивность потока во всем кластере суммарно равна  $\lambda_{\text{search,LSI}} N$ , так как для обработки одной заявки координатором, она должна быть обработана  $N$  исполнителями.

Все параметры, необходимые для расчета, кроме времени ожидания обработки подзадачи исполнителем  $W_{\text{search,LSI}}^{\text{execute}}$  и времени ожидания в очереди исполнителя  $W_{\text{search,LSI}}^{\text{execute,q}}$  известны из параметров модели. Для нахождения обеих величин необходимо вычислить время обработки заявки исполнителем  $T_{\text{search,LSI}}^{\text{execute}}$ .

Чтобы не усложнять расчеты случайностью всех процессов поиска в  $B^+$ -дереве, рассмотрены только худшие случаи поиска и обновления. Так что  $T_{\text{search,LSI}}^{\text{coordinate}}$  — это скалярная величина.

На каждом исполнителе хранится по  $d = \frac{D}{N}$  записей, так что поиск первой записи по вторичному ключу в  $B^+$ -дереве требует  $\log_m d$  операций перехода по указателям и  $\log_m d \log_2(2m - 1)$  операций сравнения ключей.

Если рассмотреть случай, когда искомый ключ является самым правым ключом листа, при этом все листья правее имеют  $\frac{2}{3}(2m - 1)$  ключей (минимальное количество ключей в узле в соответствии с определением  $B^+$ -дерева), получаем, что чтение  $s$  записей из листовых узлов  $B^+$ -дерева требует максимум  $\frac{s-1}{\frac{2}{3}(2m-1)}$  дополнительных переходов по указателям между листьями. После нахождения искомого ключа нужно перейти по указателю на полную запись в таблице.

Итого, время решения подзадачи поиска равно:

$$T_{\text{search,LSI}}^{\text{execute}} = \frac{\log_m d \log_2(2m - 1)}{f_{\text{cpu}}} + \frac{\log_m d + 3 \cdot \frac{s-1}{4m-2} + 1}{f_{\text{mem}}}. \quad (7)$$

Теперь можно найти время ожидания в очереди исполнителя  $W_{\text{search,LSI}}^{\text{execute,q}}$  и время работы над заявкой исполнителем  $W_{\text{search,LSI}}^{\text{execute}}$ . Для этого воспользуемся формулой Полячека-Хинчина [10].

Загрузка исполнителя равна:

$$\rho_{\text{search,LSI}}^{\text{executor}} = \lambda_{\text{search,LSI}}^{\text{sub}} \cdot T_{\text{search,LSI}}^{\text{execute}}. \quad (8)$$

Исполнитель справляется с нагрузкой при  $\rho_{\text{search,LSI}}^{\text{executor}} < 1$ .

По формуле Полячека—Хинчина рассчитаем среднее количество заявок в исполнителе (количество заявок в очереди и ныне обслуживающихся):

$$\begin{aligned} L_{\text{search,LSI}}^{\text{execute}} &= \rho_{\text{search,LSI}}^{\text{executor}} + \frac{\rho_{\text{search,LSI}}^{\text{executor}^2} + \lambda_{\text{search,LSI}}^{\text{sub}} D [T_{\text{search,LSI}}^{\text{execute}}]}{2(1 - \rho_{\text{search,LSI}}^{\text{executor}})} = \\ &= \rho_{\text{search,LSI}}^{\text{executor}} + \frac{\rho_{\text{search,LSI}}^{\text{executor}^2}}{2(1 - \rho_{\text{search,LSI}}^{\text{executor}})}, \end{aligned} \quad (9)$$

и среднее время ожидания заявки в исполнителе в соответствии с законом Литтла, приведенным в [9,10]:

$$W_{\text{search,LSI}}^{\text{execute}} = \frac{L_{\text{search,LSI}}^{\text{execute}}}{\lambda_{\text{search}}} = T_{\text{search,LSI}}^{\text{execute}} + \frac{\rho_{\text{search,LSI}}^{\text{executor}} T_{\text{search,LSI}}^{\text{execute}}}{2(1 - \rho_{\text{search,LSI}}^{\text{executor}})}. \quad (10)$$

Время, проведенное в очереди равно времени проведенному в системе без времени обслуживания заявки, тогда:

$$W_{\text{search,LSI}}^{\text{execute,q}} = W_{\text{search,LSI}}^{\text{execute}} - T_{\text{search,LSI}}^{\text{execute}} = \frac{\rho_{\text{search,LSI}}^{\text{executor}} T_{\text{search,LSI}}^{\text{execute}}}{2(1 - \rho_{\text{search,LSI}}^{\text{executor}})}. \quad (11)$$

Оценим время работы координатора, связанное с одним исполнителем:

$$T_{\text{search,LSI},i}^{\text{coordinate}} = t_{\text{request},i}^{\text{replicaset}} + \frac{P}{v_{\text{cluster}}} + W_{\text{search,LSI}}^{\text{execute}} + t_{\text{response},i}^{\text{replicaset}} + \frac{sr}{v_{\text{cluster}}}. \quad (12)$$

Только времена сетевой задержки  $(t_{\text{request},i}^{\text{replicaset}}, t_{\text{response},i}^{\text{replicaset}})$  — случайные величины, остальные — скалярные, так что, согласно свойствам гамма-распределения, приведенным в [14], время работы координатора на одном наборе реплик — случайная величина, имеющая сдвинутое гамма-распределение. Случайную часть  $T_{\text{search,LSI},i}^{\text{coordinate}}$  обозначим  $\mathbb{U}_i^c$ , а детерминированную —  $\mathbb{C}^c$ :

$$\mathbb{C}^c = W_{\text{search,LSI}}^{\text{execute}} + \frac{P + sr}{v_{\text{cluster}}}, \quad (13)$$

$$\mathbb{U}_i^c = T_{\text{search,LSI},i}^{\text{coordinate}} - \mathbb{C}^c \sim \Gamma(2k_{\text{cluster}}, \theta_{\text{cluster}}). \quad (14)$$

Следовательно функция распределения равна:

$$F_{\mathbb{U}_i^c}(t) = P(\mathbb{U}_i^c \leq t) = \frac{\gamma(2k_{\text{cluster}}, t/\theta_{\text{cluster}})}{\Gamma(2k_{\text{cluster}})}. \quad (15)$$

Согласно формуле (4), время работы координатора равно:

$$T_{\text{search,LSI}}^{\text{coordinate}} = \max_{i=1..N} T_{\text{search,LSI},i}^{\text{coordinate}} \quad (16)$$

случайную часть  $T_{\text{search,LSI}}^{\text{coordinate}}$  обозначим  $\mathbb{U}^c$ , максимум среди одинаковых неслучайных величин  $\mathbb{C}^c$  равен самому себе:

$$\mathbb{U}^c = T_{\text{search,LSI}}^{\text{coordinate}} - \mathbb{C}^c \quad (17)$$

значит, функция распределения  $\mathbb{U}^c$  равна:

$$F_{\mathbb{U}^c}(t) = [F_{\mathbb{U}_i^c}(t)]^N = \left[ \frac{\gamma(2k_{\text{cluster}}, t/\theta_{\text{cluster}})}{\Gamma(2k_{\text{cluster}})} \right]^N, \quad (18)$$

что позволяет нам численно найти моменты  $T_{\text{search,LSI}}^{\text{coordinate}}$  через функцию выживания [15]:

$$M[\mathbb{U}^c] = \int_0^\infty \left( 1 - \left[ \frac{\gamma(2k_{\text{cluster}}, t/\theta_{\text{cluster}})}{\Gamma(2k_{\text{cluster}})} \right]^N \right) dt, \quad (19)$$

$$M[\mathbb{U}^{c^2}] = \int_0^\infty 2t \left( 1 - \left[ \frac{\gamma(2k_{\text{cluster}}, t/\theta_{\text{cluster}})}{\Gamma(2k_{\text{cluster}})} \right]^N \right) dt, \quad (20)$$

$$M[T_{\text{search,LSI}}^{\text{coordinate}}] = M[\mathbb{U}^c] + \mathbb{C}^c, \quad (21)$$

$$D[T_{\text{search,LSI}}^{\text{coordinate}}] = D[\mathbb{U}^c] = M[\mathbb{U}^{c^2}] - (M[\mathbb{U}^c])^2. \quad (22)$$

В свою очередь:

$$M[W_{\text{search,LSI}}^{\text{coordinate}}] = W_{\text{search,LSI}}^{\text{execute,q}} + M[T_{\text{search,LSI}}^{\text{coordinate}}], \quad (23)$$

$$D[W_{\text{search,LSI}}^{\text{coordinate}}] = D[T_{\text{search,LSI}}^{\text{coordinate}}] \quad (24)$$

Время задержек в  $T_{\text{search,LSI}}^{\text{user}}$ , обозначим за  $\mathbb{U}^u$ , тогда:

$$\mathbb{U}^u = t_{\text{request}}^{\text{user}} + t_{\text{response}}^{\text{user}} \sim \Gamma(2k_{\text{user}}, \theta_{\text{user}}), \quad (25)$$

из чего можно найти математическое ожидание и дисперсию времени ожидания пользователя:

$$\begin{aligned} M[T_{\text{search,LSI}}^{\text{user}}] &= M[\mathbb{U}^u] + M[W_{\text{search,LSI}}^{\text{coordinate}}] + \frac{sNr}{v_{\text{user}}} = \\ &= k_{\text{user}}\theta_{\text{user}} + M[W_{\text{search,LSI}}^{\text{coordinate}}] + \frac{Q + sNr}{v_{\text{user}}}, \end{aligned} \quad (26)$$

$$\begin{aligned} D[T_{\text{search,LSI}}^{\text{user}}] &= D[\mathbb{U}^u] + D[W_{\text{search,LSI}}^{\text{coordinate}}] = \\ &= k_{\text{user}}\theta_{\text{user}}^2 + D[W_{\text{search,LSI}}^{\text{coordinate}}]. \end{aligned} \quad (27)$$

### 3.3.1 Расчет показателей эффективности

Максимальное количество запросов в секунду достигается при максимальной загрузке системы. Так как мы договорились, что составление плана исполнения координатором не учитывается в модели, на него можно дать бесконечную нагрузку. Тогда единственным ограничивающим фактором системы является время обработки заявки исполнителем  $T_{search,LSI}^{execute}$ .

Для устойчивости системы, нагрузка системы  $\rho_{search,LSI}^{executor}$  должна не превышать единицу. По определению, нагрузка системы рассчитывается по формуле (8), значит максимальная нагрузка системы рассчитывается по следующей формуле:

$$\lambda_{search,LSI}^{sub,max} = \frac{1}{T_{search,LSI}^{execute}}, \quad (28)$$

следовательно:

$$\lambda_{search,LSI}^{max} = \frac{1}{T_{search,LSI}^{execute}}. \quad (29)$$

Среднее время обслуживания заявки равняется  $M[T_{search,LSI}^{user}]$ .

Среднее время ожидания заявки в очереди является суммарным временем нахождения заявки во всех очередях, то есть рассчитывается по формуле (30):

$$W_{update,LSI}^{total,q} = 2W_{search,LSI}^{execute,q}, \quad (30)$$

так как сначала заявка ожидает в очереди координатора за  $W_{search,LSI}^{execute,q}$ , потом еще по столько же параллельно на каждом исполнителе.

Для вычисления вероятности необслуживания заявки (ее таймаут), нужно рассчитать вероятность того, что время обслуживания превысит максимальное время обслуживания заявки, то есть рассчитать следующее значение:

$$P(T_{search,LSI}^{user} \geq T_{timeout}). \quad (31)$$

Время ожидания пользователя  $T_{search,LSI}^{user}$  является суммой независимых случайной и детерминированной частей:



$$T_{\text{search,LSI}}^{\text{user}} = \mathbb{U}^u + W_{\text{search,LSI}}^{\text{coordinate}} + \frac{Q + sNr}{v_{\text{user}}}, \quad (32)$$

где  $W_{\text{search,LSI}}^{\text{coordinate}}$  равна  $W_{\text{search,LSI}}^{\text{execute,q}} + T_{\text{search,LSI}}^{\text{coordinate}}$ .

Обозначим детерминированную часть времени ожидания пользователя как  $\mathbb{C}^u$ . Она рассчитывается по следующей формуле:

$$\mathbb{C}^u = W_{\text{search,LSI}}^{\text{execute,q}} + \mathbb{C}^c + \frac{Q + sNr}{v_{\text{user}}}, \quad (33)$$

тогда случайная часть  $T_{\text{search,LSI}}^{\text{user}}$  равна сумме двух независимых случайных величин:

$$T_{\text{search,LSI}}^{\text{user}} - \mathbb{C}^u = \mathbb{U}^u + \mathbb{U}^c, \quad (34)$$

где  $\mathbb{U}^u \sim \Gamma(2k_{\text{user}}, \theta_{\text{user}})$  и  $\mathbb{U}^c$  — максимум  $N$  независимых одинаково распределенных случайных величин с известной функцией распределения  $F_{\mathbb{U}^c}(t)$ .

Обозначим  $\tau = T_{\text{timeout}} - \mathbb{C}^u$ , тогда:

$$P(T_{\text{search,LSI}}^{\text{user}} \geq T_{\text{timeout}}) = P(\mathbb{U}^u + \mathbb{U}^c \geq \tau). \quad (35)$$

Если  $\tau \leq 0$ , то таймаут наступает с вероятностью 1, так как детерминированная часть превышает допустимое время.

При  $\tau > 0$  воспользуемся формулой свертки. Плотность распределения  $\mathbb{U}^u$  известна в явном виде:

$$f_{\mathbb{U}^u}(t) = t^{2k_{\text{user}}-1} \frac{e^{-t/\theta_{\text{user}}}}{\theta_{\text{user}}^{2k_{\text{user}}} \Gamma(2k_{\text{user}})}, t \geq 0. \quad (36)$$

Тогда вероятность таймаута рассчитывается через свертку:

$$P(\mathbb{U}^u + \mathbb{U}^c \geq \tau) = \int_0^\tau f_{\mathbb{U}^u}(t)[1 - F_{\mathbb{U}^c}(\tau - t)]dt + \int_\tau^\infty f_{\mathbb{U}^u}(t)dt. \quad (37)$$

Первое слагаемое соответствует случаю, когда  $\mathbb{U}^u = t < \tau$ , но  $\mathbb{U}^c$  компенсирует оставшееся время до таймаута. Второе слагаемое

соответствует случаю, когда уже одной сетевой задержки пользователя достаточно для превышения таймаута.

Подставляя известные выражения:

$$\begin{aligned}
 P(T_{\text{search,LSI}}^{\text{user}} \geq T_{\text{timeout}}) = \\
 = \int_0^\tau f_{\mathbb{U}}(t) \left( 1 - \left[ \frac{\gamma(2k_{\text{cluster}}, \tau - t/\theta_{\text{cluster}})}{\Gamma(2k_{\text{cluster}})} \right]^N \right) dt + \\
 + 1 - \frac{\gamma(2k_{\text{user}}, \tau/\theta_{\text{user}})}{\Gamma(2k_{\text{user}})}.
 \end{aligned} \tag{38}$$

Данный интеграл можно вычислить численно методом Симпсона.

### 3.4 Поиск в глобальном индексе

Глобальные индексы устроены следующим образом:

- данные сегментированных таблиц распределены примерно равномерно среди наборов реплик, а принадлежность записи к набору реплик определяется значением ключа сегментирования;
- часть индекса находится на каждом наборе реплик, причем данные индекса сегментированы по вторичному ключу индексируемой таблицы.

Алгоритм поиска в кластере с такой индексной структурой выглядит следующим образом:

- 1) координатор принимает пользовательский запрос, рассчитывает на каком наборе реплик (набор реплик  $m$ ) лежит искомый вторичный ключ;
- 2) координатор отправляет запрос на поиск всех записей с искомым вторичным ключом на исполнителя набора реплик  $m$  (исполнитель  $m$ );
- 3) на исполнителе  $m$  происходит поиск всех первичных ключей и ключей сегментирования с искомым вторичным ключом в  $B^+$ -дереве;
- 4) исполнитель  $m$  отправляет на все наборы реплик, содержащие ключи сегментирования, запрос на поиск записей с первичными ключами;

- 5) на каждом таком исполнителе происходит поиск записей по значению первичных ключей в B<sup>+</sup>-дереве;
- 6) каждый исполнитель отправляет найденные записи исполнителю m;
- 7) исполнитель m объединяет данные с каждого исполнителя и отправляет найденные записи координатору;
- 8) координатор отправляет найденные записи пользователю.

На рисунке 3 представлена диаграмма последовательности описанного алгоритма. На ней изображены линии жизни следующих объектов:

- клиент — инициатор обращения к СУБД;
- координатор;
- исполнитель m — исполнитель, содержащий часть индекса с искомым вторичным ключом.
- исполнитель i — исполнитель набора реплик под номером i.

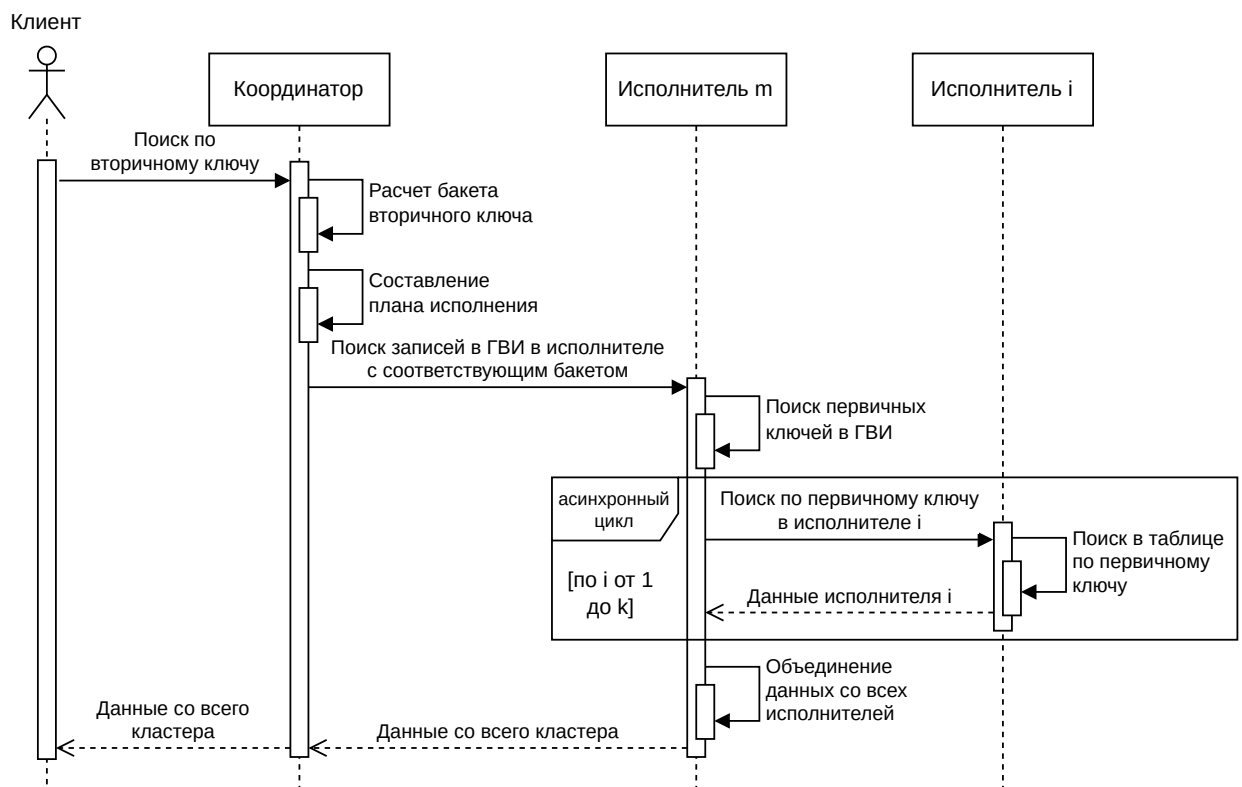


Рисунок 3 — UML Sequence диаграмма поиска в сегментированной таблице в СУБД Picodata с использованием ЛВИ

Исходя из этой диаграммы, можно оценить количество сетевых запросов при поиске с использованием ГВИ ( $NR_{\text{search}}^{\text{GSI}}$ ) по следующей формуле:

$$6 \leq NR_{\text{search}}^{\text{GSI}} \leq 2k + 4, \quad (39)$$

где  $k$  — количество наборов реплик, среди которых расположенные все найденные ключи сегментирования.

В худшем случае, если много записей сегментированной таблицы имеют одинаковое значение искомого вторичного ключа ( $K \approx 1$ ), то произойдет опрос всего кластера. Прибавим к этому отправление запросов на координатор и на исполнителя  $m$ , и получим верхнюю границу; нижняя граница получается в случае, если все найденные ключи сегментирования лежат на одном наборе реплик.

Таким образом, в сравнении с поиском при использовании ЛВИ, описанном в пункте 3.3, получается, что при правильных входных данных можно добиться значительного сокращения сетевых запросов и ускорения исполнения запросов поиска. Помимо этого, уменьшится время использования ЦПУ по всему кластеру, ввиду того, что меньшее количество исполнителей будет производить локальный поиск.

**Теорема 3.4.1** (о времени ожидания исполнения запроса координатором при поиске в ГВИ)

Время ожидания исполнения запроса координатором  $W_{\text{search,GSI}}^{\text{coordinate}}$  является случайной величиной и рассчитывается по следующей формуле:

$$\begin{aligned} W_{\text{search,GSI}}^{\text{coordinate}} = & W_{\text{search,GSI}}^{\text{execute,q}} + t_{\text{request}}^{\text{FK-execute}} + \frac{P}{v_{\text{cluster}}} + \\ & + W_{\text{search,GSI}}^{\text{FK-execute}} + t_{\text{response}}^{\text{FK-execute}} + \frac{sNr}{v_{\text{cluster}}}, \end{aligned} \quad (40)$$

где  $W_{\text{search,GSI}}^{\text{execute,q}}$  — время ожидания запроса в очереди исполнителя;  $t_{\text{request}}^{\text{FK-execute}}$  и  $t_{\text{response}}^{\text{FK-execute}}$  — времена задержки передачи запроса между координатором и исполнителем со вторичным ключом по сети;  $W_{\text{search,GSI}}^{\text{FK-execute}}$  — время ожидания ответа от исполнителя со вторичным ключом;  $P$  — размер плана исполнения;  $v_{\text{cluster}}$  — скорость передачи данных между узлами СУБД;

$s \cdot N$  — количество записей с искомым вторичным ключом в кластере;  $r$  — размер одной записи.

### Доказательство

Для начала рассмотрим время, которое проводится исключительно на самом координаторе.

Запрос сначала попадает в очередь исполнителя и находится там  $W_{\text{search,GSI}}^{\text{execute,q}}$  секунд. Далее он делегирует работу исполнителю с искомым вторичным ключом.

Отправление плана сопровождается сетевой задержкой, которая длится  $t_{\text{request}}^{\text{FK-execute}}$ , моделируемая как гамма-распределение  $\Gamma(k_{\text{cluster}}, \theta_{\text{cluster}})$ .

Положим, что план запроса весит  $P$  байтов, а скорость передачи данных между узлами кластера равна  $v_{\text{cluster}}$  байтов в секунду. Тогда весь план будет передан за  $P/v_{\text{cluster}}$  секунд.

На исполнителе время ожидания исполнения заявки, то есть время, проведенное в очереди в сумме с временем непосредственной обработки заявки обозначим как  $W_{\text{search,GSI}}^{\text{execute}}$ .

Результат работы каждого исполнителя отправляется координатору отдельным сетевым запросом с задержкой  $t_{\text{request},i}^{\text{replicaset}}$ , моделируемой гамма-распределением  $\Gamma(k_{\text{cluster}}, \theta_{\text{cluster}})$ .

Исходя из предположения, что данные распределены равномерно (в том числе по вторичным ключам) по наборам реплик, то на каждом исполнителе находится  $s$ , равное  $\frac{D}{NK}$  искомым записей. Положим, что одна запись весит  $r$  байтов, тогда передача данных с всего кластера занимает  $\frac{sNr}{v_{\text{cluster}}}$  секунд.

Итого, время ожидания исполнения запроса координатором  $W_{\text{search,GSI}}^{\text{coordinate}}$  рассчитывается по следующей формуле:

$$\begin{aligned}
 W_{\text{search,GSI}}^{\text{coordinate}} = & W_{\text{search,GSI}}^{\text{execute,q}} + t_{\text{request}}^{\text{FK-execute}} + \frac{P}{v_{\text{cluster}}} + \\
 & + W_{\text{search,GSI}}^{\text{FK-execute}} + t_{\text{response}}^{\text{FK-execute}} + \frac{sNr}{v_{\text{cluster}}},
 \end{aligned} \tag{41}$$

В правой части равенства присутствуют случайные величины, так что и левая часть является случайной величиной. ■

Рассмотрим из чего состоит время ожидания ответа от исполнителя с искомым вторичным ключом.

**Теорема 3.4.2** (о времени ожидания ответа от исполнителя с искомым вторичным ключом)

Время ожидания ответа от исполнителя с искомым вторичным ключом  $W_{\text{search,GSI}}^{\text{FK-execute}}$  является случайной величиной и рассчитывается по следующей формуле:

$$\begin{aligned} W_{\text{search,GSI}}^{\text{FK-execute}} &= W_{\text{search,GSI}}^{\text{execute,q}} + T_{\text{search,GSI}}^{\text{FK-execute}} + \max_{i=1..k} [T_{\text{search,GSI},i}^{\text{FK-execute}}] = \\ &= W_{\text{search,GSI}}^{\text{execute,q}} + T_{\text{search,GSI}}^{\text{FK-execute}} + \\ &+ \max_{i=1..k} \left[ t_{\text{request},i}^{\text{replicaset}} + \frac{P_i}{v_{\text{cluster}}} + W_{\text{search,GSI}}^{\text{execute}} + t_{\text{response},i}^{\text{replicaset}} + \frac{s_i r}{v_{\text{cluster}}} \right], \end{aligned} \quad (42)$$

где  $W_{\text{search,GSI}}^{\text{execute,q}}$  — время ожидания запроса в очереди исполнителя;  $T_{\text{search,GSI}}^{\text{FK-execute}}$  — время поиска первичных ключей в исполнителе со вторичными ключами;  $T_{\text{search,GSI},i}^{\text{FK-execute}}$  — время ожидания ответа от исполнителя под номером  $i$ ;  $t_{\text{request},i}^{\text{replicaset}}$  и  $t_{\text{response},i}^{\text{replicaset}}$  — времена задержки передачи запроса между координатором и исполнителем под номером  $i$  по сети;  $P_i$  — размер плана исполнения для исполнителя под номером  $i$ ;  $v_{\text{cluster}}$  — скорость передачи данных между узлами СУБД;  $W_{\text{search,GSI}}^{\text{execute}}$  — время ожидания обработки запроса исполнителем;  $s_i$  — количество записей с искомым вторичным ключом на наборе реплик;  $r$  — размер одной записи;  $k \leq N$  — количество наборов реплик, на которых располагаются первичные ключи.

### Доказательство

Для начала рассмотрим время, которое проводится исключительно на самом исполнителе со вторичными ключами.

Запрос сначала попадает в очередь исполнителя и находится там  $W_{\text{search,GSI}}^{\text{execute,q}}$  секунд. После этого происходит поиск вторичных ключей в B<sup>+</sup>-дереве за  $T_{\text{search,GSI}}^{\text{FK-execute}}$ .

Итого, на координаторе заявка обрабатывается  $W_{\text{search,GSI}}^{\text{execute,q}} + T_{\text{search,GSI}}^{\text{FK-execute}}$  секунд, далее он делегирует работу исполнителям. Рассмотрим сколько времени занимает обработка поиска на исполнителях.

Каждому исполнителю составляется отдельный план запроса  $P_i$ . Он отправляется каждому исполнителю по сети, которых всего  $k$  штук — ключей сегментирования может быть меньше чем наборов реплик в кластере, и несколько ключей сегментирования может быть на одном наборе реплик. Мы будем моделировать  $k$  как случайную величину.

Каждое отправление плана сопровождается сетевой задержкой, которая длится  $t_{\text{request},i}^{\text{replicaset}}$ , моделируемая как гамма-распределение  $\Gamma(k_{\text{cluster}}, \theta_{\text{cluster}})$ .

Положим, что план запроса весит  $P_i$  байтов, а скорость передачи данных между узлами кластера равна  $v_{\text{cluster}}$  байтов в секунду. Тогда весь план будет передан через  $P/v_{\text{cluster}}$  секунд.

На исполнителе время ожидания исполнения заявки, то есть время, проведенное в очереди в сумме с временем непосредственной обработки заявки обозначим как  $W_{\text{search,GSI}}^{\text{execute}}$ .

Результат работы каждого исполнителя отправляется исполнителю со вторичными ключами отдельным сетевым запросом с задержкой  $t_{\text{request},i}^{\text{replicaset}}$ , моделируемой гамма-распределением  $\Gamma(k_{\text{cluster}}, \theta_{\text{cluster}})$ .

На каждом исполнителе находится  $s_i$  записей. Одна запись весит  $r$  байтов, тогда передача данных с одного исполнителя занимает  $\frac{s_i r}{v_{\text{cluster}}}$  секунд. Итого, время, которое занимает прием и обработка плана исполнения на одном исполнителе равна следующей сумме:

$$t_{\text{request},i}^{\text{replicaset}} + \frac{P_i}{v_{\text{cluster}}} + W_{\text{search,GSI}}^{\text{execute}} + t_{\text{response},i}^{\text{replicaset}} + \frac{s_i r}{v_{\text{cluster}}}, \quad (43)$$

которую мы обозначим  $T_{\text{search,GSI},i}^{\text{FK-execute}}$ .

Исполнитель со вторичным ключом должен дожидаться ответа от всех исполнителей, чтобы вернуть объединенный результат пользователю, так что время ожидания исполнителей равно времени ожидания самого долгого исполнителя, что является максимумом среди всех  $T_{\text{search,GSI},i}^{\text{FK-execute}}$ .

Итого, время ожидания исполнения запроса исполнителем со вторичным ключом  $W_{\text{search,GSI}}^{\text{FK-execute}}$  рассчитывается по следующей формуле:

$$\begin{aligned}
W_{\text{search,GSI}}^{\text{FK-execute}} &= W_{\text{search,GSI}}^{\text{execute,q}} + T_{\text{search,GSI}}^{\text{FK-execute}} + \max_{i=1..k} [T_{\text{search,GSI},i}^{\text{FK-execute}}] = \\
&= W_{\text{search,GSI}}^{\text{execute,q}} + T_{\text{search,GSI}}^{\text{FK-execute}} + \\
&+ \max_{i=1..k} \left[ t_{\text{request},i}^{\text{replicaset}} + \frac{P_i}{v_{\text{cluster}}} + W_{\text{search,GSI}}^{\text{execute}} + t_{\text{response},i}^{\text{replicaset}} + \frac{s_i r}{v_{\text{cluster}}} \right],
\end{aligned} \tag{44}$$

В правой части равенства присутствуют случайные величины, так что и левая часть является случайной величиной. ■

Так как один пользовательский запрос превращается в  $k$  запросов для исполнителей, то интенсивность потока, поступающих в исполнителей рассчитывается по следующей формуле:

$$\lambda_{\text{search,GSI}}^{\text{sub}} = \frac{\lambda_{\text{search,GSI}} k}{N}. \tag{45}$$

Рассчитаем чему равно  $T_{\text{search,GSI}}^{\text{execute}}$ . Поиск  $n$  вразнобой-идущих ключей занимает в  $n$  раз дольше времени, чем поиск одного ключа, то есть совершается  $n \log_m d$  переходов по указателю и  $n \log_m d \log_2(2m - 1)$  операций сравнения.

Тогда,  $T_{\text{search,GSI}}^{\text{execute}}$  рассчитывается по следующей формуле:

$$T_{\text{search,GSI}}^{\text{execute}} = \frac{n \log_m d \log_2(2m - 1)}{f_{\text{cpu}}} + \frac{n \log_m d}{f_{\text{mem}}} \tag{46}$$

и является детерминированной величиной.

Отсюда нагрузка на исполнителя вычисляется по следующей формуле:

$$\rho_{\text{search,GSI}}^{\text{executor}} = T_{\text{search,GSI}}^{\text{execute}} \lambda_{\text{search,GSI}}^{\text{sub}}. \tag{47}$$

Для вычисления времени обслуживания заявки и времени ожидания в очереди вновь воспользуемся формулой Полячека-Хинчина по аналогии с формулой (9):



$$W_{\text{search,GSI}}^{\text{execute}} = T_{\text{search,GSI}}^{\text{execute}} + \frac{\rho_{\text{search,GSI}}^{\text{executor}} T_{\text{search,GSI}}^{\text{execute}}}{2(1 - \rho_{\text{search,GSI}}^{\text{executor}})}, \quad (48)$$

$$W_{\text{search,GSI}}^{\text{execute,q}} = \frac{\rho_{\text{search,GSI}}^{\text{executor}} T_{\text{search,GSI}}^{\text{execute}}}{2(1 - \rho_{\text{search,GSI}}^{\text{executor}})}. \quad (49)$$

Оценим время работы исполнителя со вторичным ключом, связанное с одним исполнителем:

$$T_{\text{search,GSI},i}^{\text{FK-execute}} = t_{\text{request},i}^{\text{replicaset}} + \frac{P_i}{v_{\text{cluster}}} + W_{\text{search,GSI}}^{\text{execute}} + t_{\text{response},i}^{\text{replicaset}} + \frac{s_i r}{v_{\text{cluster}}}. \quad (50)$$

Для удобства, возьмем средние значения для плана исполнения и найденного количества данных, то есть:

$$P_i = P, \quad (51)$$

$$s_i = s, \quad (52)$$

тогда, формула (50) примет вид:

$$T_{\text{search,GSI},i}^{\text{FK-execute}} = t_{\text{request},i}^{\text{replicaset}} + \frac{P}{v_{\text{cluster}}} + W_{\text{search,GSI}}^{\text{execute}} + t_{\text{response},i}^{\text{replicaset}} + \frac{sr}{v_{\text{cluster}}}. \quad (53)$$

Случайную часть  $T_{\text{search,GSI},i}^{\text{FK-execute}}$  обозначим  $\mathbb{U}_i^{\text{FK}}$ , а детерминированную —  $\mathbb{C}_i^{\text{FK}}$ :

$$\mathbb{C}_i^{\text{FK}} = W_{\text{search,GSI}}^{\text{execute}} + \frac{P + sr}{v_{\text{cluster}}}, \quad (54)$$

$$\mathbb{U}_i^{\text{FK}} = T_{\text{search,GSI},i}^{\text{FK-execute}} - \mathbb{C}_i^{\text{FK}} \sim \Gamma(2k_{\text{cluster}}, \theta_{\text{cluster}}). \quad (55)$$

Следовательно функция распределения равна:

$$F_{\mathbb{U}_i^{\text{FK}}}(t) = P(\mathbb{U}_i^{\text{FK}} \leq t) = \frac{\gamma(2k_{\text{cluster}}, t/\theta_{\text{cluster}})}{\Gamma(2k_{\text{cluster}})}. \quad (56)$$

Дальнейшие рассуждения по поводу  $T_{\text{search,GSI}}^{\text{FK-execute}}$  аналогичны рассуждениям для  $T_{\text{search,LSI}}^{\text{coordinate}}$ , за исключением добавления времени поиска в В+\*-дереве и взятии максимума не из  $N$  значений, а из  $k$ .

Случайную часть  $T_{\text{search,GSI}}^{\text{FK-execute}}$  обозначим  $\mathbb{U}^{\text{FK}}$ , неслучайную часть обозначим  $\mathbb{C}^{\text{FK}}$ . Максимум среди одинаковых неслучайных величин  $\mathbb{C}_i^{\text{FK}}$  равен самому себе:

$$\mathbb{C}^{\text{FK}} = \mathbb{C}_i^{\text{FK}} + T_{\text{search,GSI}}^{\text{FK-execute}} \quad (57)$$

$$\mathbb{U}^{\text{FK}} = T_{\text{search,GSI}}^{\text{FK-execute}} - \mathbb{C}^{\text{FK}} \quad (58)$$

значит, функция распределения  $\mathbb{U}^{\text{FK}}$  равна:

$$F_{\mathbb{U}^{\text{FK}}}(t) = [F_{\mathbb{U}_i^{\text{FK}}}(t)]^k = \left[ \frac{\gamma(2k_{\text{cluster}}, t/\theta_{\text{cluster}})}{\Gamma(2k_{\text{cluster}})} \right]^k, \quad (59)$$

что позволяет нам численно найти моменты  $T_{\text{search,GSI}}^{\text{FK-execute}}$  через функцию выживания [15]:

$$M[\mathbb{U}^{\text{FK}}] = \int_0^\infty \left( 1 - \left[ \frac{\gamma(2k_{\text{cluster}}, t/\theta_{\text{cluster}})}{\Gamma(2k_{\text{cluster}})} \right]^k \right) dt, \quad (60)$$

$$M[\mathbb{U}^{\text{FK}^2}] = \int_0^\infty 2t \left( 1 - \left[ \frac{\gamma(2k_{\text{cluster}}, t/\theta_{\text{cluster}})}{\Gamma(2k_{\text{cluster}})} \right]^k \right) dt, \quad (61)$$

$$M[T_{\text{search,GSI}}^{\text{FK-execute}}] = M[\mathbb{U}^{\text{FK}}] + \mathbb{C}^{\text{FK}}, \quad (62)$$

$$D[T_{\text{search,GSI}}^{\text{FK-execute}}] = D[\mathbb{U}^{\text{FK}}] = M[\mathbb{U}^{\text{FK}^2}] - (M[\mathbb{U}^{\text{FK}}])^2. \quad (63)$$

В свою очередь:

$$M[W_{\text{search,GSI}}^{\text{FK-execute}}] = W_{\text{search,GSI}}^{\text{execute,q}} + M[T_{\text{search,GSI}}^{\text{FK-execute}}], \quad (64)$$

$$D[W_{\text{search,GSI}}^{\text{FK-execute}}] = D[T_{\text{search,GSI}}^{\text{FK-execute}}] \quad (65)$$

Время задержек в  $T_{\text{search,GSI}}^{\text{coordinate}}$ , обозначим за  $\mathbb{U}^{\text{c}}$ , тогда:

$$\mathbb{U}^{\text{c}} = t_{\text{request}}^{\text{FK-execute}} + t_{\text{response}}^{\text{FK-execute}} \sim \Gamma(2k_{\text{cluster}}, \theta_{\text{cluster}}), \quad (66)$$

из чего можно найти математическое ожидание и дисперсию времени работы координатора:

$$\begin{aligned}
M[T_{\text{search,GSI}}^{\text{coordinate}}] &= M[\mathbb{U}^c] + M[W_{\text{search,GSI}}^{\text{FK-execute}}] + \frac{P + srN}{v_{\text{user}}} = \\
&= k_{\text{cluster}} \theta_{\text{cluster}} + M[W_{\text{search,GSI}}^{\text{FK-execute}}] + \frac{P + srN}{v_{\text{user}}},
\end{aligned} \tag{67}$$

$$\begin{aligned}
D[T_{\text{search,GSI}}^{\text{coordinate}}] &= D[\mathbb{U}^c] + D[W_{\text{search,GSI}}^{\text{FK-execute}}] = \\
&= k_{\text{cluster}} \theta_{\text{cluster}}^2 + D[W_{\text{search,LSI}}^{\text{coordinate}}].
\end{aligned} \tag{68}$$

Время обработки заявки координатором равно:

$$W_{\text{search,GSI}}^{\text{coordinate}} = W_{\text{search,GSI}}^{\text{execute,q}} + T_{\text{search,GSI}}^{\text{coordinate}} \tag{69}$$

Наконец, можно найти среднее и дисперсию времени ожидания пользователем. Для этого обозначим за  $\mathbb{U}^u$  время сетевых задержек:

$$\mathbb{U}^u = t_{\text{request}}^{\text{user}} + t_{\text{response}}^{\text{user}} \sim \Gamma(2k_{\text{user}}, \theta_{\text{user}}), \tag{70}$$

следовательно:

$$\begin{aligned}
M[T_{\text{search,GSI}}^{\text{user}}] &= M[\mathbb{U}^u] + M[W_{\text{search,GSI}}^{\text{coordinate}}] + \frac{P + srN}{v_{\text{cluster}}} = \\
&= k_{\text{user}} \theta_{\text{user}} + M[W_{\text{search,GSI}}^{\text{coordinate}}] + \frac{P + srN}{v_{\text{cluster}}},
\end{aligned} \tag{71}$$

$$\begin{aligned}
D[T_{\text{search,GSI}}^{\text{user}}] &= D[\mathbb{U}^u] + D[W_{\text{search,GSI}}^{\text{coordinate}}] = \\
&= k_{\text{user}} \theta_{\text{user}}^2 + D[W_{\text{search,GSI}}^{\text{coordinate}}].
\end{aligned} \tag{72}$$

### 3.4.1 Расчет показателей эффективности

Максимальное количество запросов в секунду достигается при максимальной загрузке системы. Единственным ограничивающим фактором системы является время обработки заявки исполнителем  $T_{\text{search,LSI}}^{\text{execute}}$ .

Для устойчивости системы, загрузка системы  $\rho_{\text{search,GSI}}^{\text{executor}}$  должна не превышать единицу. По определению, загрузка системы рассчитывается по формуле (47), значит максимальная нагрузка системы рассчитывается по следующей формуле:

$$\lambda_{\text{search,GSI}}^{\text{sub,max}} = \frac{1}{T_{\text{search,GSI}}^{\text{execute}}}, \quad (73)$$

а по формуле (45), верно, что:

$$\lambda_{\text{search,GSI}}^{\text{max}} = \frac{\lambda_{\text{search,GSI}}^{\text{sub,max}} N}{k} = \frac{N}{T_{\text{search,GSI}}^{\text{execute}} k}. \quad (74)$$

Среднее время обслуживания заявки равняется  $M[T_{\text{search,GSI}}^{\text{user}}]$ .

Среднее время ожидания заявки в очереди является суммарным временем нахождения заявки во всех очередях, то есть рассчитывается по формуле (75):

$$W_{\text{search,GSI}}^{\text{total,q}} = 3W_{\text{search,GSI}}^{\text{execute,q}}, \quad (75)$$

так как сначала заявка ожидает в очереди координатора за  $W_{\text{search,GSI}}^{\text{execute,q}}$ , потом еще столько же в исполнителе с искомым вторичным ключом и еще по столько же параллельно на каждом исполнителе.

Для вычисления вероятности таймаута заявки необходимо найти  $P(T_{\text{search,GSI}}^{\text{user}} \geq T_{\text{timeout}})$ .

Время ожидания пользователя  $T_{\text{search,GSI}}^{\text{user}}$  является суммой независимых случайных и детерминированных частей. Выделим полную детерминированную часть:

$$\mathbb{C}^u = W_{\text{search,GSI}}^{\text{execute,q}} + \frac{P + srN}{v_{\text{cluster}}} + W_{\text{search,GSI}}^{\text{execute,q}} + \mathbb{C}^{\text{FK}} + \frac{P + srN}{v_{\text{user}}}. \quad (76)$$

Случайная часть  $T_{\text{search,GSI}}^{\text{user}}$  равна сумме трёх независимых случайных величин:

$$T_{\text{search,GSI}}^{\text{user}} - \mathbb{C}^u = \mathbb{U}^u + \mathbb{U}^c + \mathbb{U}^{\text{FK}}, \quad (77)$$

где  $\mathbb{U}^u \sim \Gamma(2k_{\text{user}}, \theta_{\text{user}})$  — сетевые задержки между пользователем и координатором;  $\mathbb{U}^c \sim \Gamma(2k_{\text{cluster}}, \theta_{\text{cluster}})$  — сетевые задержки между координатором и исполнителем со вторичным ключом;  $\mathbb{U}^{\text{FK}}$  — максимум  $k$  независимых одинаково распределённых случайных величин с функцией распределения  $F_{\mathbb{U}^{\text{FK}}}(t)$ .

Обозначим  $\tau = T_{\text{timeout}} - \mathbb{C}^u$ . Тогда:

$$P(T_{\text{search,GSI}}^{\text{user}} \geq T_{\text{timeout}}) = P(\mathbb{U}^u + \mathbb{U}^c + \mathbb{U}^{\text{FK}} \geq \tau). \quad (78)$$

При  $\tau > 0$  заметим, что сумма  $\mathbb{U}^u + \mathbb{U}^c$  двух независимых гамма-распределенных величин не сводится к гамма-распределению, так как их параметры различны в общем случае. Поэтому воспользуемся последовательной сверткой.

Сначала найдем распределение суммы  $\mathbb{U}^u + \mathbb{U}^c$ . Плотности обеих величин известны:

$$f_{\mathbb{U}^u}(t) = t^{2k_{\text{user}}-1} \frac{e^{-t/\theta_{\text{user}}}}{\theta_{\text{user}}^{2k_{\text{user}}} \Gamma(2k_{\text{user}})}, t \geq 0, \quad (79)$$

$$f_{\mathbb{U}^c}(t) = t^{2k_{\text{cluster}}-1} \frac{e^{-t/\theta_{\text{cluster}}}}{\theta_{\text{cluster}}^{2k_{\text{cluster}}} \Gamma(2k_{\text{cluster}})}, t \geq 0. \quad (80)$$

Плотность их суммы  $\mathbb{U}^{\text{uc}} = \mathbb{U}^u + \mathbb{U}^c$  вычисляется свёрткой:

$$f_{\mathbb{U}^{\text{uc}}}(t) = \int_0^t f_{\mathbb{U}^u}(u) f_{\mathbb{U}^c}(t-u) du, t \geq 0. \quad (81)$$

Теперь вероятность таймаута выражается через свёртку  $\mathbb{U}^{\text{uc}}$  и  $\mathbb{U}^{\text{FK}}$ :

$$P(\mathbb{U}^{\text{uc}} + \mathbb{U}^{\text{FK}} \geq \tau) = \int_0^\tau f_{\mathbb{U}^{\text{uc}}}(t) [1 - F_{\mathbb{U}^{\text{FK}}}(\tau - t)] dt + \int_\tau^\infty f_{\mathbb{U}^{\text{uc}}}(t) dt. \quad (82)$$

Подставляя выражение для  $F_{\mathbb{U}^{\text{FK}}}$ :

$$\begin{aligned} P(T_{\text{search,GSI}}^{\text{user}} \geq T_{\text{timeout}}) = \\ = \int_0^\tau f_{\mathbb{U}^{\text{uc}}}(t) \left( 1 - \left[ \frac{\gamma(2k_{\text{cluster}}, \tau - t/\theta_{\text{cluster}})}{\Gamma(2k_{\text{cluster}})} \right]^k \right) dt + 1 - F_{\mathbb{U}^{\text{uc}}}(\tau), \end{aligned} \quad (83)$$

где  $F_{\mathbb{U}^{\text{uc}}}(\tau) = \int_0^\tau f_{\mathbb{U}^{\text{uc}}}(t) dt$  — функция распределения суммы  $\mathbb{U}^{\text{uc}}$ .

Все интегралы можно вычислить численно методом Симпсона. При этом внутренняя свёртка для  $f_{\mathbb{U}^{\text{uc}}}(t)$  также вычисляется численно в каждой точке, что приводит к двойному численному интегрированию.

### 3.5 Обновление в локальном индексе

Алгоритм записи в кластере с локальным вторичным индексом выглядит следующим образом:

- 1) координатор принимает пользовательский запрос, рассчитывает бакет обновляемой записи и отправляет план исполнения соответствующему исполнителю;
- 2) исполнитель добавляет информацию об обновлении таблицы в журнал упреждающей записи, обновляет  $B^+$ -дерево индекса первичного ключа и обновляет  $B^+$ -дерево вторичного индекса;
- 3) исполнитель отправляет подтверждение записи координатору;
- 4) координатор отправляет подтверждение записи пользователю.

На рисунке 4 представлена диаграмма последовательности обновления ЛВИ. На ней изображены линии жизни следующих объектов:

- клиент — инициатор обращения к СУБД Picodata;
- координатор;
- исполнитель  $m$  — исполнитель набора реплик на котором лежит первичный ключ записи.

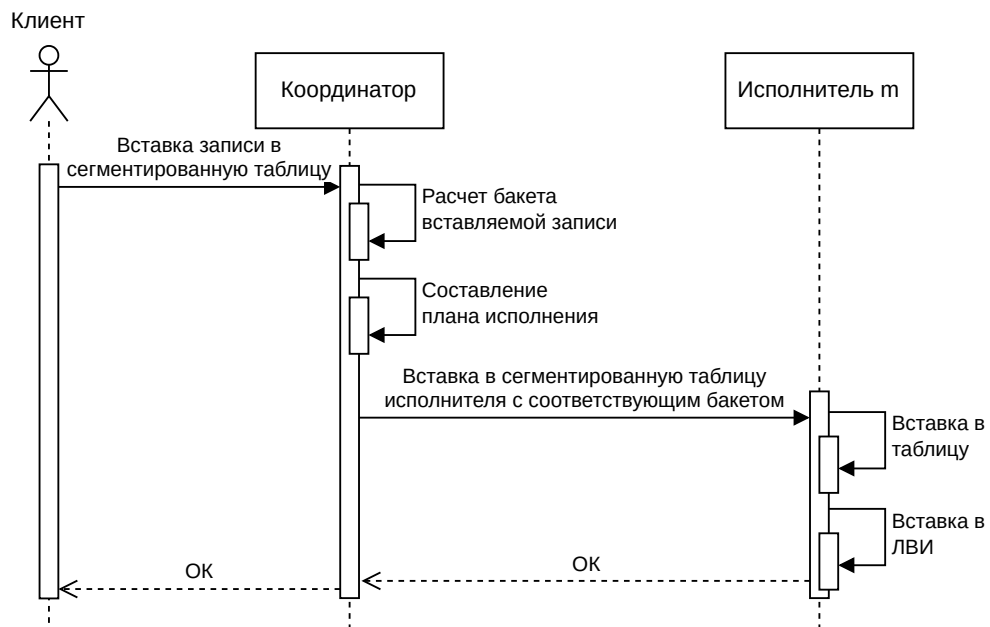


Рисунок 4 — UML Sequence диаграмма построения ЛВИ в СУБД Picodata при вставке в сегментированную таблицу

Количество сетевых запросов при обновлении ЛВИ ( $NR_{update}^{LSI}$ ) равно 4.

**Теорема 3.5.1** (о времени ожидания исполнения запроса координатором при обновлении ЛВИ)

Время ожидания исполнения запроса координатором  $W_{\text{update,LSI}}^{\text{coordinate}}$  является случайной величиной и рассчитывается по следующей формуле:

$$\begin{aligned} W_{\text{update,LSI}}^{\text{coordinate}} &= W_{\text{update,LSI}}^{\text{execute,q}} + T_{\text{update,LSI}}^{\text{coordinate}} = \\ &= W_{\text{update,LSI}}^{\text{execute,q}} + t_{\text{request}}^{\text{replicaset}} + \frac{P}{v_{\text{cluster}}} + W_{\text{update,LSI}}^{\text{execute}} + t_{\text{response}}^{\text{replicaset}}, \end{aligned} \quad (84)$$

где  $W_{\text{update,LSI}}^{\text{execute,q}}$  — время ожидания запроса в очереди исполнителя;  $T_{\text{update,LSI}}^{\text{coordinate}}$  — время ожидания ответа от координатора;  $t_{\text{request}}^{\text{replicaset}}$  и  $t_{\text{response}}^{\text{replicaset}}$  — времена задержки передачи запроса между координатором и исполнителем по сети;  $P$  — размер плана исполнения;  $v_{\text{cluster}}$  — скорость передачи данных между узлами СУБД;  $W_{\text{update,LSI}}^{\text{execute}}$  — время ожидания обработки запроса исполнителем.

### Доказательство

Запрос сначала попадает в очередь исполнителя и находится там  $W_{\text{search,LSI}}^{\text{execute,q}}$  секунд.

Составленный план запроса отправляется одному исполнителю по сети. Отправление плана сопровождается сетевой задержкой, которая длится  $t_{\text{request}}^{\text{replicaset}}$ , моделируемая как гамма-распределение  $\Gamma(k_{\text{cluster}}, \theta_{\text{cluster}})$ .

Положим, что план запроса весит  $P$  байтов, а скорость передачи данных между узлами кластера равна  $v_{\text{cluster}}$  байтов в секунду. Тогда весь план будет передан через  $P/v_{\text{cluster}}$  секунд.

На исполнителе время ожидания исполнения заявки, то есть время, проведенное в очереди в сумме с временем непосредственной обработки заявки обозначим как  $W_{\text{search,LSI}}^{\text{execute}}$ .

Результат работы исполнителя отправляется координатору отдельным сетевым запросом с задержкой  $t_{\text{request},i}^{\text{replicaset}}$ , моделируемой гамма-распределением  $\Gamma(k_{\text{cluster}}, \theta_{\text{cluster}})$ .

Сам результат весит 16 байтов — это подтверждение записи, время его передачи мало, поэтому не учитывается.

Итого, время ожидания исполнения запроса координатора  $W_{\text{update,LSI}}^{\text{coordinate}}$  рассчитывается по следующей формуле:

$$\begin{aligned} W_{\text{update,LSI}}^{\text{coordinate}} &= W_{\text{update,LSI}}^{\text{execute,q}} + T_{\text{update,LSI}}^{\text{coordinate}} = \\ &= W_{\text{update,LSI}}^{\text{execute,q}} + t_{\text{request}}^{\text{replicaset}} + \frac{P}{v_{\text{cluster}}} + W_{\text{update,LSI}}^{\text{execute}} + t_{\text{response}}^{\text{replicaset}}, \end{aligned} \quad (85)$$

В правой части равенства присутствуют случайные величины, так что и левая часть тоже является случайной величиной. ■

Получается, каждая заявка на обновление порождает ровно одну подзадачу для одного исполнителя. Поток запросов координатора  $\lambda_{\text{update,LSI}}$  распределяется равномерно на исполнителей:

$$\lambda_{\text{update,LSI}}^{\text{sub}} = \frac{\lambda_{\text{update}}}{N}. \quad (86)$$

Рассмотрим чему равно время обработки заявки исполнителем  $W_{\text{update,LSI}}^{\text{execute}}$ .

Обновление начинается с записи в журнал упреждающей записи, которое занимает постоянное время  $T_{\text{WAL}}$ . После этого происходит поиск позиции ключа в B<sup>+</sup>-дереве для обновления, на что тратится  $\log_m d \log_2(2m - 1)$  операций сравнения ключей и  $\log_m d$  переходов по указателям.

После поиска происходит обновление дерева, например вставка нового элемента. В лучшем случае, когда лист не заполнен до конца, вставка происходит за одну операцию записи в запоминающее устройство, в худшем — все узлы в пути до листа (включая сам лист) нужно разделить на два. Для простоты модели, сошлемся на статью [16], в которой доказано, что средняя заполненность B<sup>\*</sup>-дерева, а значит и средняя заполненность каждого узла равна 0.81, где 0 — узел пуст, 1 — узел полон.

Получается, что в среднем остается  $(2m - 1)(1 - 0.81) = 0.19(2m - 1)$  вставок перед переполнением узла, соответственно вероятность переполнения на конкретной вставке равна  $(0.19(2m - 1))^{-1} \approx \frac{5.26}{2m-1}$ . Разделение узла происходит за 3 операции записи — перезапись разделенного узла, запись нового братского узла и перезапись родительского узла. Случай каскадного разделения родительских узлов



возможен, но маловероятен (вероятность этого события уменьшается с каждым родителем экспоненциально), так что он не учтен.

Итого, время решения подзадачи обновления равно:

$$T_{\text{update,LSI}}^{\text{execute}} = T_{\text{WAL}} + 2 \left( \frac{\log_m d \log_2(2m-1)}{f_{\text{cpu}}} + \frac{\log_m d + 3 \cdot \frac{5.26}{2m-1}}{f_{\text{mem}}} \right) \quad (87)$$

Это также детерминированная величина. Вычислим время ожидания обработки заявки с помощью формулы Полячека-Хинчина и время, проведенное в очереди исполнителя:

$$W_{\text{update,LSI}}^{\text{execute}} = T_{\text{update,LSI}}^{\text{execute}} + \frac{\rho_{\text{update,LSI}}^{\text{executor}} T_{\text{update,LSI}}^{\text{execute}}}{2(1 - \rho_{\text{update,LSI}}^{\text{executor}})}, \quad (88)$$

$$W_{\text{update,LSI}}^{\text{execute,q}} = \frac{\rho_{\text{update,LSI}}^{\text{executor}} T_{\text{update,LSI}}^{\text{execute}}}{2(1 - \rho_{\text{update,LSI}}^{\text{executor}})}. \quad (89)$$

Рассчитаем время работы координатора:

$$T_{\text{update,LSI}}^{\text{coordinate}} = t_{\text{request},i}^{\text{replicaset}} + \frac{P}{v_{\text{cluster}}} + W_{\text{search,LSI}}^{\text{execute}} + t_{\text{response},i}^{\text{replicaset}} + \frac{sr}{v_{\text{cluster}}}. \quad (90)$$

Только времена сетевой задержки — случайные величины, остальные — скалярные, так что, согласно свойствам гамма-распределения, время работы координатора — случайная величина, имеющая сдвинутое гамма-распределение. Случайную часть  $T_{\text{update,LSI}}^{\text{coordinate}}$  обозначим  $\mathbb{U}^c$ , а детерминированную —  $\mathbb{C}^c$ :

$$\mathbb{C}^c = W_{\text{update,LSI}}^{\text{execute}} + \frac{P + sr}{v_{\text{cluster}}}, \quad (91)$$

$$\mathbb{U}^c = T_{\text{update,LSI}}^{\text{coordinate}} - \mathbb{C}^c \sim \Gamma(2k_{\text{cluster}}, \theta_{\text{cluster}}). \quad (92)$$

Следовательно функция распределения равна:

$$F_{\mathbb{U}^c}(t) = P(\mathbb{U}^c \leq t) = \frac{\gamma(2k_{\text{cluster}}, t/\theta_{\text{cluster}})}{\Gamma(2k_{\text{cluster}})}. \quad (93)$$

Времена сетевых задержек во времени ожидания пользователя  $T_{\text{update,LSI}}^{\text{user}}$  обозначим  $\mathbb{U}^u$ , тогда:

$$\mathbb{U}^u = t_{\text{request}}^{\text{user}} + t_{\text{response}}^{\text{user}} \sim \Gamma(2k_{\text{user}}, \theta_{\text{user}}), \quad (94)$$

следовательно:

$$\begin{aligned} M[T_{\text{update,LSI}}^{\text{user}}] &= M[\mathbb{U}^u] + M[W_{\text{update,LSI}}^{\text{coordinate}}] = \\ &= k_{\text{user}} \theta_{\text{user}} + M[W_{\text{update,LSI}}^{\text{coordinate}}], \end{aligned} \quad (95)$$

$$\begin{aligned} D[T_{\text{update,LSI}}^{\text{user}}] &= D[\mathbb{U}^c] + D[W_{\text{update,LSI}}^{\text{coordinate}}] = \\ &= k_{\text{user}} \theta_{\text{user}}^2 + D[T_{\text{update,LSI}}^{\text{coordinate}}]. \end{aligned} \quad (96)$$

### 3.6 Расчет параметров эффективности

Максимальное количество запросов в секунду достигается при максимальной загрузке системы. Единственным ограничивающим фактором системы является время обработки заявки исполнителем  $T_{\text{update,LSI}}^{\text{execute}}$ .

Для устойчивости системы, нагрузка системы  $\rho_{\text{update,LSI}}^{\text{executor}}$  должна не превышать единицу. Максимальная нагрузка системы рассчитывается по следующей формуле:

$$\lambda_{\text{update,LSI}}^{\text{sub,max}} = \frac{1}{T_{\text{update,LSI}}^{\text{execute}}}, \quad (97)$$

а по формуле (86), верно, что:

$$\lambda_{\text{update,LSI}}^{\text{max}} = \lambda_{\text{update,LSI}}^{\text{sub,max}} N = \frac{N}{T_{\text{update,LSI}}^{\text{execute}}}. \quad (98)$$

Среднее время обслуживания заявки равняется  $M[T_{\text{update,LSI}}^{\text{user}}]$ .

Среднее время ожидания заявки в очереди является суммарным временем нахождения заявки во всех очередях, то есть рассчитывается по формуле (99):

$$W_{\text{update,LSI}}^{\text{total,q}} = 2W_{\text{update,LSI}}^{\text{execute,q}}, \quad (99)$$

так как сначала заявка ожидает в очереди координатора за  $W_{\text{update,LSI}}^{\text{execute,q}}$ , потом еще столько же в исполнителе.

Найдем вероятность того, что время ожидания пользователем завершения операции обновления  $T_{\text{update,LSI}}^{\text{user}}$  превысит заданный лимит  $T_{\text{timeout}}$ , то есть вычислим  $P(T_{\text{update,LSI}}^{\text{user}} \geq T_{\text{timeout}})$ .

Время ожидания пользователя складывается из детерминированной и случайной частей:

$$T_{\text{update,LSI}}^{\text{user}} = W_{\text{update,LSI}}^{\text{execute,q}} + \mathbb{C}^c + \mathbb{U}^c + \mathbb{U}^u, \quad (100)$$

где  $\mathbb{U}^u \sim \text{Gamma}(2k_{\text{user}}, \theta_{\text{user}})$  — суммарная задержка сети между пользователем и координатором;  $\mathbb{U}^c \sim \text{Gamma}(2k_{\text{cluster}}, \theta_{\text{cluster}})$  — суммарная задержка сети между координатором и исполнителем; детерминированная составляющая  $\mathbb{C}^u = W_{\text{update,LSI}}^{\text{execute,q}} + \mathbb{C}^c$ .

Определим  $\tau = T_{\text{timeout}} - \mathbb{C}^u$ . При  $\tau > 0$  воспользуемся формулой свёртки для суммы двух независимых случайных величин. Плотность  $\mathbb{U}^u$  известна:

$$f_{\mathbb{U}^u}(t) = \frac{t^{2k_{\text{user}}-1} e^{-t/\theta_{\text{user}}}}{\theta_{\text{user}}^{2k_{\text{user}}} \Gamma(2k_{\text{user}})}, t \geq 0. \quad (101)$$

Функция распределения  $\mathbb{U}^c$  выражается через регуляризованную нижнюю неполную гамма-функцию:

$$F_{\mathbb{U}^c}(t) = P(\mathbb{U}^c \leq t) = \frac{\gamma(2k_{\text{cluster}}, t/\theta_{\text{cluster}})}{\Gamma(2k_{\text{cluster}})}. \quad (102)$$

Тогда, вероятность таймаута рассчитывается по следующей формуле:

$$\begin{aligned}
P(T_{\text{update,LSI}}^{\text{user}} \geq T_{\text{timeout}}) &= P(\mathbb{U}^u + \mathbb{U}^c \geq \tau) = \\
&= \int_0^\tau f_{\mathbb{U}^u}(t)(1 - F_{\mathbb{U}^c}(\tau - t))dt + \int_\tau^\infty f_{\mathbb{U}^u}(t)dt.
\end{aligned} \tag{103}$$

Подставляя явные выражения, получаем окончательную формулу:

$$\begin{aligned}
P(T_{\text{update,LSI}}^{\text{user}} \geq T_{\text{timeout}}) &= \\
&= \int_0^\tau \frac{t^{2k_{\text{user}}-1} e^{-t/\theta_{\text{user}}}}{\theta_{\text{user}}^{2k_{\text{user}}} \Gamma(2k_{\text{user}})} \left( 1 - \frac{\gamma(2k_{\text{cluster}}, (\tau - t)/\theta_{\text{cluster}})}{\Gamma(2k_{\text{cluster}})} \right) dt + \\
&\quad + 1 - \frac{\gamma(2k_{\text{user}}, \tau/\theta_{\text{user}})}{\Gamma(2k_{\text{user}})}.
\end{aligned} \tag{104}$$

Все величины, входящие в формулу, определены в предыдущих разделах. Интеграл вычисляется численно методом Симпсона.

### 3.7 Обновление в глобальном индексе

Алгоритм записи в кластере с такой индексной структурой выглядит следующим образом:

- 1) координатор принимает пользовательский запрос, рассчитывает бакет обновляемой записи и отправляет план исполнения исполнителю с ключом сегментирования;
- 2) исполнитель с ключом сегментирования добавляет информацию об обновлении таблицы в журнал упреждающей записи, обновляет В+\*-дерево индекса первичного ключа и отправляет подтверждение записи координатору;
- 3) координатор рассчитывает бакет вторичного ключа и отправляет план исполнения исполнителю со вторичным ключом;
- 4) исполнитель со вторичным ключом добавляет информацию об обновлении таблицы в журнал упреждающей записи, обновляет В+\*-дерево ГВИ;
- 5) исполнитель со вторичным ключом возвращает координатору подтверждение записи;
- 6) координатор отправляет подтверждение записи пользователю.

На рисунке 5 представлена диаграмма последовательности обновления ГВИ. На ней изображены линии жизни следующих объектов:

- клиент — инициатор обращения к СУБД Picodata;
- координатор;
- исполнитель m — исполнитель набора реплик, на котором хранится ключ сегментирования записи.
- исполнитель k — исполнитель набора реплик, на котором хранится вторичный ключ записи.

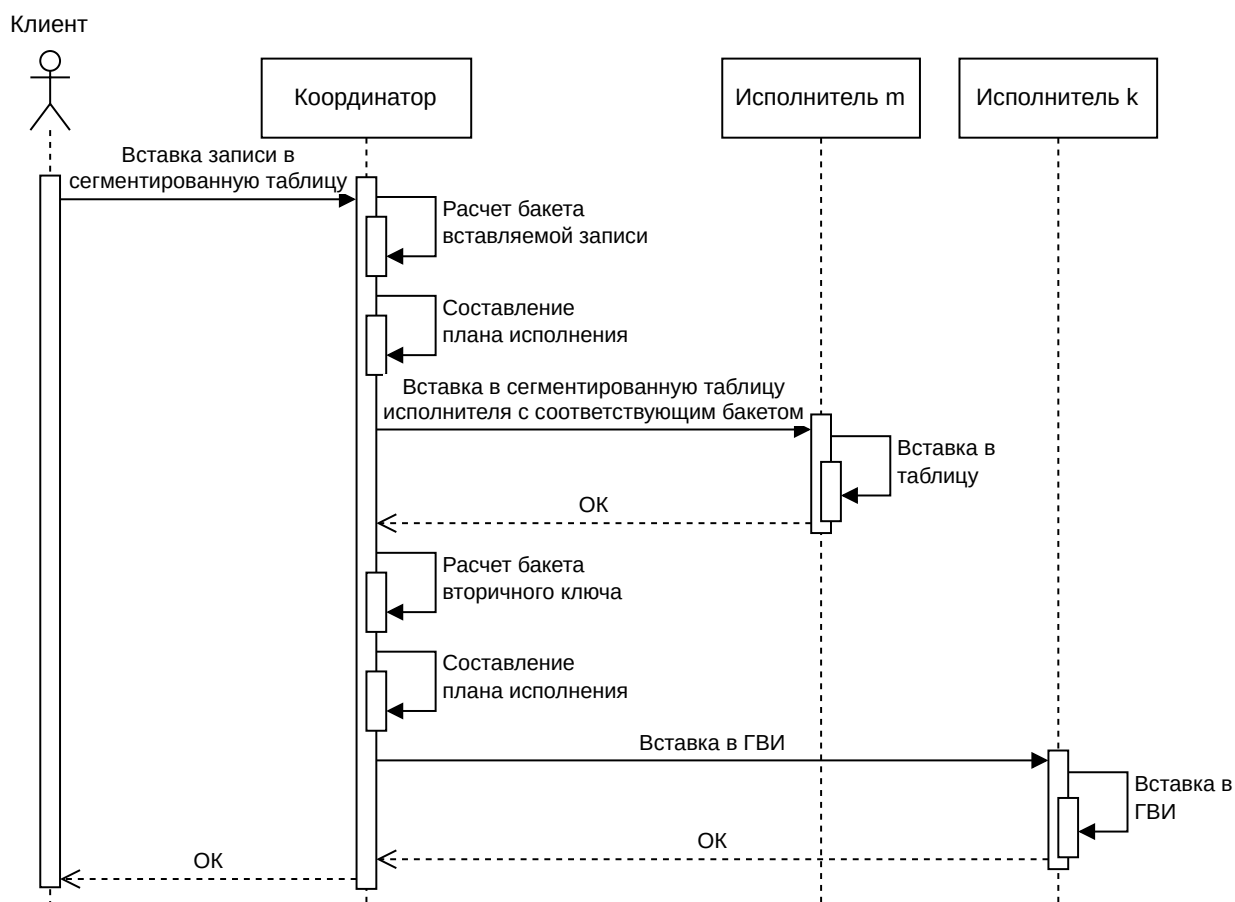


Рисунок 5 — UML Sequence диаграмма построения ГВИ в СУБД Picodata при вставке в сегментированную таблицу

Количество сетевых запросов при обновлении ГВИ ( $NR_{update}^{GSI}$ ) равно 4.

**Теорема 3.7.1** (о времени ожидания исполнения запроса координатором при обновлении ГВИ)

Время ожидания исполнения запроса координатором  $W_{update, GSI}^{coordinate}$  является случайной величиной и рассчитывается по следующей формуле:

$$\begin{aligned}
W_{\text{update,GSI}}^{\text{coordinate}} &= W_{\text{update,GSI}}^{\text{execute,q}} + T_{\text{update,GSI}}^{\text{SK-coordinate}} + T_{\text{update,GSI}}^{\text{FK-coordinate}} = W_{\text{update,GSI}}^{\text{execute,q}} + \\
&+ t_{\text{request}}^{\text{SK-replicaset}} + \frac{P}{v_{\text{cluster}}} + W_{\text{SK-update,GSI}}^{\text{execute}} + t_{\text{response}}^{\text{SK-replicaset}} + \\
&+ t_{\text{request}}^{\text{FK-replicaset}} + \frac{P}{v_{\text{cluster}}} + W_{\text{FK-update,GSI}}^{\text{execute}} + t_{\text{response}}^{\text{FK-replicaset}},
\end{aligned} \tag{105}$$

где  $W_{\text{update,GSI}}^{\text{execute,q}}$  — время ожидания запроса в очереди исполнителя;  $T_{\text{update,GSI}}^{\text{SK-coordinate}}$  — время ожидания ответа от исполнителя с ключом сегментирования;  $T_{\text{update,GSI}}^{\text{FK-coordinate}}$  — время ожидания ответа от исполнителя со вторичным ключом;  $t_{\text{request}}^{\text{SK-replicaset}}$  и  $t_{\text{response}}^{\text{SK-replicaset}}$  — времена задержки передачи запроса между координатором и исполнителем с ключом сегментирования по сети;  $t_{\text{request}}^{\text{FK-replicaset}}$  и  $t_{\text{response}}^{\text{FK-replicaset}}$  — времена задержки передачи запроса между координатором и исполнителем со вторичным ключом по сети;  $P$  — размер плана исполнения;  $v_{\text{cluster}}$  — скорость передачи данных между узлами СУБД;  $W_{\text{SK-update,GSI}}^{\text{execute}}$  — время ожидания обработки запроса исполнителем с ключом сегментирования;  $W_{\text{FK-update,GSI}}^{\text{execute}}$  — время ожидания обработки запроса исполнителем со вторичным ключом.

### Доказательство

Запрос сначала попадает в очередь исполнителя и находится там  $W_{\text{search,GSI}}^{\text{execute,q}}$  секунд.

Составленный план запроса отправляется исполнителю с первичным ключом по сети. Все сетевые запросы сопровождается сетевой задержкой, моделируемая как гамма-распределение  $\Gamma(k_{\text{cluster}}, \theta_{\text{cluster}})$ .

Положим, что план запроса весит  $P$  байтов, а скорость передачи данных между узлами кластера равна  $v_{\text{cluster}}$  байтов в секунду. Тогда весь план будет передан через  $P/v_{\text{cluster}}$  секунд.

На исполнителе с ключом сегментирования время ожидания исполнения заявки обозначим как  $W_{\text{SK-update,GSI}}^{\text{execute}}$ . На исполнителе со вторичным ключом время ожидания исполнения заявки обозначим как  $W_{\text{FK-update,GSI}}^{\text{execute}}$ .

Сам результат весит 16 байтов — это подтверждение записи, время его передачи мало, поэтому не учитывается.

Итого, время ожидания исполнения запроса координатора  $W_{\text{update,GSI}}^{\text{coordinate}}$  рассчитывается по следующей формуле:

$$\begin{aligned}
W_{\text{update,GSI}}^{\text{coordinate}} = & W_{\text{update,GSI}}^{\text{execute,q}} + T_{\text{update,GSI}}^{\text{SK-coordinate}} + T_{\text{update,GSI}}^{\text{FK-coordinate}} = W_{\text{update,GSI}}^{\text{execute,q}} + \\
& + t_{\text{request}}^{\text{SK-replicaset}} + \frac{P}{v_{\text{cluster}}} + W_{\text{SK-update,GSI}}^{\text{execute}} + t_{\text{response}}^{\text{SK-replicaset}} + \\
& + t_{\text{request}}^{\text{FK-replicaset}} + \frac{P}{v_{\text{cluster}}} + W_{\text{FK-update,GSI}}^{\text{execute}} + t_{\text{response}}^{\text{FK-replicaset}}, \quad (106)
\end{aligned}$$

В правой части равенства присутствуют случайные величины, так что и левая часть является случайной величиной. ■

Каждая заявка на обновление порождает ровно две подзадачи. Поток запросов координатора  $\lambda_{\text{update,GSI}}$  распределяется равномерно на исполнителей:

$$\lambda_{\text{update,GSI}}^{\text{sub}} = \frac{2\lambda_{\text{update}}}{N}. \quad (107)$$

Время исполнения запроса на исполнителе с ключом сегментирования рассчитывается как худший случай поиска в  $B^+$ -дереве (уже несколько раз рассмотренный) в сумме с временем записи в журнал упреждающей записи. Пропустим повторные вычисления:

$$\begin{aligned}
T_{\text{update,GSI}}^{\text{SK-execute}} = & T_{\text{WAL}} + \frac{\log_m d \log_2(2m-1)}{f_{\text{cpu}}} + \\
& + \frac{\log_m d + 3 \cdot \frac{5.26}{2m-1}}{f_{\text{mem}}} \quad (108)
\end{aligned}$$

Подобный расчет верен и для исполнителя со вторичным ключом:

$$\begin{aligned}
T_{\text{update,GSI}}^{\text{FK-execute}} = & T_{\text{WAL}} + \frac{\log_m d \log_2(2m-1)}{f_{\text{cpu}}} + \\
& + \frac{\log_m d + 3 \cdot \frac{5.26}{2m-1}}{f_{\text{mem}}} \quad (109)
\end{aligned}$$

Расчеты приведенные ниже верны как для исполнителя с ключами сегментирования, так и для исполнителя со вторичными ключами.

Вычислим время ожидания обработки заявки и время, проведенное в очереди, с помощью формулы Полячека-Хинчина:

$$W_{\text{SK-update,GSI}}^{\text{execute}} = T_{\text{update,GSI}}^{\text{SK-execute}} + \frac{\rho_{\text{update,GSI}}^{\text{executor}} T_{\text{update,GSI}}^{\text{SK-execute}}}{2(1 - \rho_{\text{update,GSI}}^{\text{executor}})}. \quad (110)$$

$$W_{\text{update,GSI}}^{\text{execute,q}} = \frac{\rho_{\text{update,GSI}}^{\text{executor}} T_{\text{update,GSI}}^{\text{SK-execute}}}{2(1 - \rho_{\text{update,GSI}}^{\text{executor}})}. \quad (111)$$

Рассчитаем время работы координатора. Только времена сетевой задержки — случайные величины, остальные — скалярные, так что время работы координатора — случайная величина, имеющая сдвинутое гамма-распределение. Случайную часть  $T_{\text{update,GSI}}^{\text{SK-coordinate}}$  обозначим  $\mathbb{U}^c$ , а детерминированную —  $\mathbb{C}^c$ :

$$\mathbb{C}^c = W_{\text{SK-update,GSI}}^{\text{execute}} + W_{\text{FK-update,GSI}}^{\text{execute}} + \frac{P}{v_{\text{cluster}}}, \quad (112)$$

$$\mathbb{U}^c = T_{\text{update,GSI}}^{\text{SK-coordinate}} - \mathbb{C}^c \sim \Gamma(4k_{\text{cluster}}, \theta_{\text{cluster}}). \quad (113)$$

Следовательно функция распределения равна:

$$F_{\mathbb{U}^c}(t) = P(\mathbb{U}^c \leq t) = \frac{\gamma(4k_{\text{cluster}}, t/\theta_{\text{cluster}})}{\Gamma(4k_{\text{cluster}})}. \quad (114)$$

Времена сетевых задержек во времени ожидания пользователя  $T_{\text{update,GSI}}^{\text{user}}$  обозначим  $\mathbb{U}^u$ , тогда:

$$\mathbb{U}^u = t_{\text{request}}^{\text{user}} + t_{\text{response}}^{\text{user}} \sim \Gamma(2k_{\text{user}}, \theta_{\text{user}}), \quad (115)$$

следовательно:

$$\begin{aligned} M[T_{\text{update,GSI}}^{\text{user}}] &= M[\mathbb{U}^u] + M[W_{\text{update,GSI}}^{\text{coordinate}}] = \\ &= k_{\text{user}} \theta_{\text{user}} + 2W_{\text{update,GSI}}^{\text{execute,q}} + 2M[T_{\text{update,GSI}}^{\text{SK-coordinate}}], \end{aligned} \quad (116)$$

$$\begin{aligned} D[T_{\text{update,GSI}}^{\text{user}}] &= D[\mathbb{U}^c] + D[W_{\text{update,GSI}}^{\text{coordinate}}] = \\ &= k_{\text{user}} \theta_{\text{user}}^2 + 2D[T_{\text{update,GSI}}^{\text{SK-coordinate}}]. \end{aligned} \quad (117)$$



### 3.8 Расчет параметров эффективности

Максимальное количество запросов в секунду достигается при максимальной загрузке системы. Так как мы договорились, что по факту работа на координаторе не учитывается в модели, то на него можно подать бесконечную нагрузку. Тогда единственным ограничивающим фактором системы является время обработки заявки исполнителем  $T_{\text{update,GSI}}^{\text{SK-execute}}$ .

Для устойчивости системы, загрузка системы  $\rho_{\text{update,GSI}}^{\text{executor}}$  должна не превышать единицу, значит максимальная нагрузка системы рассчитывается по следующей формуле:

$$\lambda_{\text{update,GSI}}^{\text{sub,max}} = \frac{1}{T_{\text{update,GSI}}^{\text{SK-execute}}}, \quad (118)$$

а по формуле (107), верно, что:

$$\lambda_{\text{update,GSI}}^{\text{max}} = \frac{\lambda_{\text{update,GSI}}^{\text{sub,max}} N}{2} = \frac{N}{2T_{\text{update,GSI}}^{\text{SK-execute}}}. \quad (119)$$

Среднее время обслуживания заявки равняется  $M[T_{\text{update,GSI}}^{\text{user}}]$ .

Среднее время ожидания заявки в очереди является суммарным временем нахождения заявки во всех очередях, то есть рассчитывается по формуле (120):

$$W_{\text{update,GSI}}^{\text{total,q}} = 3W_{\text{update,GSI}}^{\text{execute,q}}, \quad (120)$$

так как заявка ожидает в очереди координатора  $W_{\text{update,GSI}}^{\text{execute,q}}$  секунд, потом еще столько же в исполнителе с ключом сегментирования и столько же в исполнителе со вторичным ключом.

Найдем вероятность того, что время ожидания пользователем завершения операции обновления превысит заданный лимит  $T_{\text{timeout}}$ , то есть вычислим  $P(T_{\text{update,GSI}}^{\text{user}} \geq T_{\text{timeout}})$ .

Время ожидания пользователя складывается из детерминированной и случайной частей:

$$T_{\text{update,GSI}}^{\text{user}} = W_{\text{update,GSI}}^{\text{execute,q}} + \mathbb{C}^c + \mathbb{U}^c + \mathbb{U}^u, \quad (121)$$

где

- $\mathbb{U}^u \sim \text{Gamma}(2k_{\text{user}}, \theta_{\text{user}})$  — суммарная задержка сети между пользователем и координатором (запрос + ответ);
- $\mathbb{U}^c \sim \text{Gamma}(4k_{\text{cluster}}, \theta_{\text{cluster}})$  — суммарная задержка сети между координатором и исполнителем;
- детерминированная составляющая  $\mathbb{C}^u = W_{\text{update, GSI}}^{\text{execute, q}} + \mathbb{C}^c$ .

Результат подтверждения записи пренебрежимо мал, поэтому не учитывается.

Определим  $\tau = T_{\text{timeout}} - \mathbb{C}^u$ . При  $\tau > 0$  воспользуемся формулой свёртки для суммы двух независимых случайных величин. Обозначим  $X = \mathbb{U}^u$ ,  $Y = \mathbb{U}^c$ . Плотность  $X$  известна:

$$f_{X(t)} = \frac{t^{2k_{\text{user}}-1} e^{-t/\theta_{\text{user}}}}{\theta_{\text{user}}^{2k_{\text{user}}} \Gamma(2k_{\text{user}})}, t \geq 0. \quad (122)$$

Функция распределения  $Y$  выражается через регуляризованную нижнюю неполную гамма-функцию:

$$F_{Y(y)} = P(Y \leq y) = \frac{\gamma(4k_{\text{cluster}}, y/\theta_{\text{cluster}})}{\Gamma(4k_{\text{cluster}})}. \quad (123)$$

Вероятность таймаута:

$$\begin{aligned} P(T_{\text{update, GSI}}^{\text{user}} \geq T_{\text{timeout}}) &= P(X + Y \geq \tau) = \\ &= \int_0^\tau f_{X(t)} (1 - F_{Y(\tau-t)}) dt + \int_\tau^\infty f_{X(t)} dt. \end{aligned} \quad (124)$$

Второе слагаемое — вероятность того, что одна лишь пользовательская задержка уже превышает  $\tau$ , и равно  $1 - P(2k_{\text{user}}, \tau/\theta_{\text{user}})$ , где  $P(a, x) = \gamma(a, x)/\Gamma(a)$ . Первое слагаемое учитывает случаи, когда  $X = t < \tau$ , но  $Y$  компенсирует оставшееся время.

Подставляя явные выражения, получаем окончательную формулу:

$$\begin{aligned}
P(T_{\text{update,GSI}}^{\text{user}} \geq T_{\text{timeout}}) = \\
= \int_0^\tau \frac{t^{2k_{\text{user}}-1} e^{-t/\theta_{\text{user}}}}{\theta_{\text{user}}^{2k_{\text{user}}} \Gamma(2k_{\text{user}})} \left( 1 - \frac{\gamma(4k_{\text{cluster}}, (\tau - t)/\theta_{\text{cluster}})}{\Gamma(4k_{\text{cluster}})} \right) dt + \\
+ 1 - \frac{\gamma(2k_{\text{user}}, \tau/\theta_{\text{user}})}{\Gamma(2k_{\text{user}})}. \quad (125)
\end{aligned}$$

Все величины, входящие в формулу, определены в предыдущих разделах. Интеграл вычисляется численно методом Симпсона, так как замкнутая форма для свертки гамма-распределений с разными параметрами масштаба отсутствует. В частном случае  $\theta_{\text{user}} = \theta_{\text{cluster}}$  сумма  $X + Y$  имеет гамма-распределение  $\Gamma(2k_{\text{user}} + 4k_{\text{cluster}}, \theta_{\text{user}})$ , и вероятность можно вычислить аналитически через регуляризованную гамма-функцию.

### 3.9 Результат расчетов и сравнение

В данной главе сведены воедино результаты анализа ЛВИ и ГВИ в СУБД Picodata, полученные в пунктах 3.3-3.7. Сравнение проводится по трем направлениям: качественная оценка архитектурных свойств, количественное сопоставление формул показателей эффективности и анализ предельных режимов работы.

#### 3.9.1 Качественное сравнение

Качественные различия двух индексных структур определяются способом размещения данных индекса и порождаемыми алгоритмами опроса кластера.

Поиск по вторичным ключам:

- в ЛВИ индекс сегментирован вместе с данными, поэтому для поиска по вторичному ключу координатор вынужден опросить все наборы реплик. Число сетевых запросов линейно зависит от размера кластера:  $\text{NR}_{\text{search}}^{\text{LSI}} = 2N + 2$ , согласно формуле (3). Это ограничивает горизонтальное масштабирование при поисковых нагрузках;
- в ГВИ индекс сегментирован по самому вторичному ключу, благодаря чему координатор направляет запрос единственному исполнителю, владеющему нужной частью индекса. Число сетевых запросов лежит в диапазоне от 6 до  $2N + 4$ , согласно формуле (39).

При высокой кардинальности вторичного ключа ( $K \approx D$ ) сетевых взаимодействий становится значительно меньше, чем в ЛВИ.

Обновление индекса:

- в ЛВИ обновление затрагивает только тот набор реплик, которому принадлежит модифицируемая запись. Число сетевых запросов равно 4;
- в ГВИ обновление требует двухэтапной фиксации: сначала обновляется таблица и первичный индекс на одном наборе реплик, затем — индекс на другом. Возникают две подзадачи, дважды выполняется запись в журнал упреждающей записи, а число сетевых запросов равно 6, но полное время операции возрастает.

Важно отметить, что сейчас ввиду отсутствия распределенных транзакций в СУБД Picodata, строгая консистенция невозможна при обновлении ГВИ, в отличие от ЛВИ. С введением такого механизма консистенция будет достигаться за счет дольшего времени обработки заявки.

### 3.9.2 Количественное сравнение

Количественное сопоставление проводится на основе итоговых выражений для показателей эффективности СМО из четырех предыдущих разделов. Сравниваются:

- максимальная интенсивность входного потока  $\lambda^{\max}$ , при которой система остается устойчивой;
- среднее время ожидания пользователя  $M[T^{\text{user}}]$ ;
- среднее суммарное время в очередях  $W^{\text{total},q}$ ;
- вероятность таймаута  $P(T^{\text{user}} \geq T_{\text{timeout}})$ .

#### 3.9.2.1 Максимальная пропускная способность

Устойчивость каждой конфигурации определяется загрузкой исполнителей. Ниже приведены максимальные интенсивности запросов, которые способен выдержать кластер из  $N$  наборов реплик.

Таблица 2 — Максимальные интенсивности запросов, при которых сохраняется устойчивость

| Конфигурация | ЛВИ                                                                                     | ГВИ                                                                                         |
|--------------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Поиск        | $\lambda_{\text{search,LSI}}^{\max} = \frac{1}{T_{\text{search,LSI}}^{\text{execute}}}$ | $\lambda_{\text{search,GSI}}^{\max} = \frac{N}{kT_{\text{search,GSI}}^{\text{execute}}}$    |
| Обновление   | $\lambda_{\text{update,LSI}}^{\max} = \frac{N}{T_{\text{update,LSI}}^{\text{execute}}}$ | $\lambda_{\text{update,GSI}}^{\max} = \frac{N}{2T_{\text{update,GSI}}^{\text{SK-execute}}}$ |

Сравним пропускную способность поиска. Отношение максимальных интенсивностей:

$$\frac{\lambda_{\text{search,GSI}}^{\max}}{\lambda_{\text{search,LSI}}^{\max}} \approx \left( \frac{N}{k} \right). \quad (126)$$

Таким образом, ГВИ не уступает ЛВИ даже при самом худшем варианте — когда  $k = N$ , а в остальных случаях показывает себя лучше.

Пропускная способность обновления в ГВИ примерно вдвое ниже, чем в ЛВИ, из-за записи в двух исполнителей:

$$\lambda_{\text{update,GSI}}^{\max} \approx \left( \frac{1}{2} \right) \lambda_{\text{update,LSI}}^{\max}. \quad (127)$$

### 3.9.2.2 Среднее время ожидания пользователя

Среднее время  $M[T^{\text{user}}]$  складывается из сетевых задержек, времени работы координатора и времени ожидания в очередях:

- время поиска в ЛВИ включает ожидание ответа от всех  $N$  исполнителей и максимум их времен обслуживания. С ростом  $N$  максимум растет как  $O(\log N)$  (для распределения с легким хвостом), а детерминированная часть пропорциональна  $\frac{1}{N}$  по числу найденных записей на узел;
- в ГВИ поиск ожидает ответа от  $k$  исполнителей, причем на координаторе дополнительно ожидается работа исполнителя со вторичным ключом индекса. При малых  $k$  среднее время поиска в ГВИ существенно ниже, при больших  $k$  оно приближается к времени ЛВИ из-за двойного посещения очередей.

Обновление в ЛВИ всегда затрагивает только один набор реплик, поэтому его среднее время минимально. В ГВИ суммарное время состоит из двух последовательных взаимодействий с разными наборами реплик, что дает приблизительно удвоенное время по сравнению с ЛВИ при одинаковых интенсивностях, так как именно сетевые задержки становятся узким горлышком.

### 3.9.2.3 Среднее время в очередях

### 3.9.2.4 Вероятность таймаута

Вероятность превышения лимита  $T_{\text{timeout}}$  выражается интегралами свертки:

- в ЛВИ случайная составляющая времени пользователя складывается из задержек пользователь–кластер и кластер–один исполнитель, поскольку координатор ждет максимум, но сетевых обменов с каждым исполнителем всего два — дисперсия максимума  $N$  одинаковых величин меньше дисперсии суммы дополнительных этапов ГВИ;
- в ГВИ случайная часть включает задержки пользователь–координатор, координатор–владелец индекса и максимум  $k$  обменов с исполнителями — суммирование трех независимых компонент увеличивает дисперсию, что при одинаковом среднем может повышать вероятность таймаута.

### 3.9.3 Выводы и область применения

Проведенный анализ позволяет сформулировать условия, при которых глобальный вторичный индекс предпочтительнее локального, и наоборот.

ГВИ дает выигрыш для поиска при следующих условиях:

- высокая кардинальность вторичного ключа, так что  $k$  мало (в идеале  $k \approx 1$ ) — в этом случае время поиска и число сетевых взаимодействий значительно снижаются;
- кластер содержит много узлов — ЛВИ опрашивает все узлы, тогда как ГВИ — лишь часть;

- сеть между узлами нестабильна или обладает высокой латентностью, тогда сокращение числа сетевых обменов критически важно.

ЛВИ сохраняет преимущество в следующих ситуациях:

- кардинальность вторичного ключа мала;
- в кластере немного узлов — дополнительная нагрузка при записи себя не оправдывает;
- доминирует нагрузка на обновление данных — ГВИ требует двухэтапной записи с дополнительными сетевыми задержками;

Таким образом, выбор индексной структуры не является универсальным и должен опираться на ожидаемый профиль нагрузки, объем данных и топологию кластера. Приведенные в главах 3.3-3.7 математические модели и полученные выше формулы позволяют выполнить численный расчет показателей эффективности для конкретных значений параметров и принять обоснованное архитектурное решение.

## 4 Модель функционирования плагина

### 4.1 Контекстная диаграмма

На рисунке 6 представлена контекстная диаграмма разрабатываемого плагина.

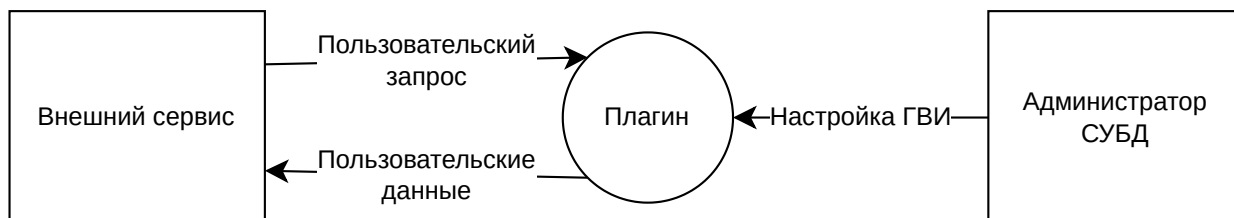


Рисунок 6 — Контекстная диаграмма разрабатываемого плагина

Проектируемым объектом диаграммы является плагин добавления функционала ГВИ в СУБД Picodata, он обозначен кругом с надписью «Плагин». Взаимодействующие с проектируемым объектом элементы окружения обозначены прямоугольниками, потоки данных между ними и проектируемым объектом обозначены стрелками.

На контекстной диаграмме представлены следующие элементы окружения:

- внешний сервис — приложение, использующее СУБД. Его потоки данных:
  - вход: пользовательский запрос — запрос на поиск по вторичному ключу или вставку в таблицу; выход: пользовательские данные — искомые записи или подтверждение записи;
- администратор СУБД — лицо, настраивающее СУБД. Его поток данных:
  - вход: настройка данных.

Диаграмма выполнена в нотации, описанной в статьях [17,18].

### 4.2 Диаграмма потоков данных

Выделенными прямоугольниками изображены являются внешние сущности по отношению к разрабатываемому плагину. Скругленными прямоугольниками показаны процессы — функции обработки данных.



Прямоугольниками с острыми углами показаны хранилища, которыми являются таблицы в СУБД.

На рисунке 7 представлена диаграмма потоков данных между внешним сервисом, использующим СУБД, и плагином. Красными стрелками показан поток данных при поиске, голубыми — при обновлении.

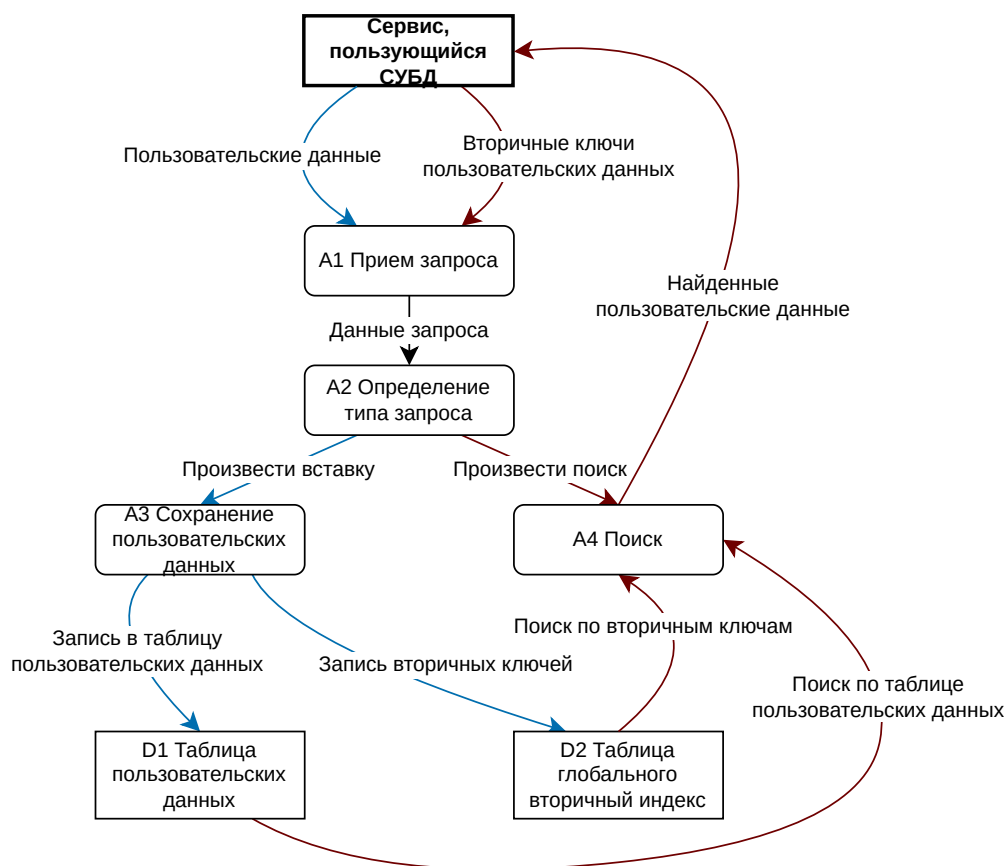


Рисунок 7 — Диаграмма потоков данных использования плагина ГВИ внешним сервисом

Диаграмма содержит следующие процессы:

- А1 Прием запроса — чтение запроса пользователя, отправленных в СУБД;
- А2 Определение типа запроса — превращение данных запроса пользователя в тип запроса с полезной информацией;
- А3 Сохранение пользовательских данных — обновление данных таблицы и индексов в СУБД;
- А4 Поиск — нахождение записи в СУБД по фильтрам из запроса пользователя.

Диаграмма содержит следующие хранилища данных:

- D1 Таблица пользовательских данных — проиндексированная таблица СУБД с записями пользователей;
- D2 Таблица глобального вторичного индекса — таблица СУБД, хранящая глобальный вторичный индекс, построенный над таблицей с записями пользователей;

На рисунке 8 представлена диаграмма потоков данных между администратором СУБД и плагином. Зелеными стрелками показан поток данных при построении индекса, синими — при удалении.

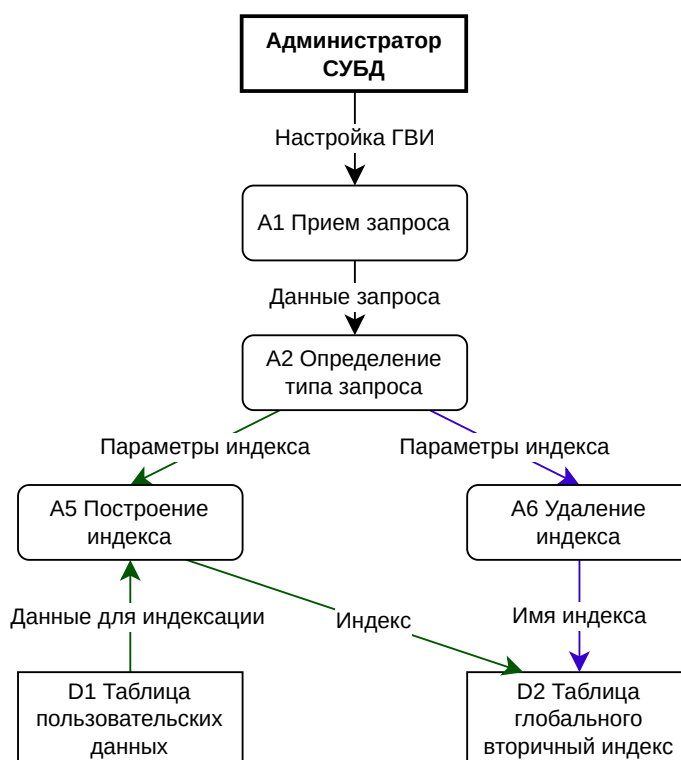


Рисунок 8 — Диаграмма потоков данных использования плагина ГВИ администратором СУБД

Помимо упомянутых выше, диаграмма содержит следующие процессы:

- A5 Построение индекса — индексирование таблицы, указанной администратором;
- A6 Удаление индекса — прекращение индексирования пользовательской таблицы и удаление таблицы глобального вторичного индекса.

### 4.2.1 Функциональная декомпозиция

На рисунке 9 представлена функциональная декомпозиция плагина ГВИ. Каждый процесс с прошлых диаграмм разбит на конкретные подфункции. Это необходимо для иерархического разделения плагина на функциональные модули.

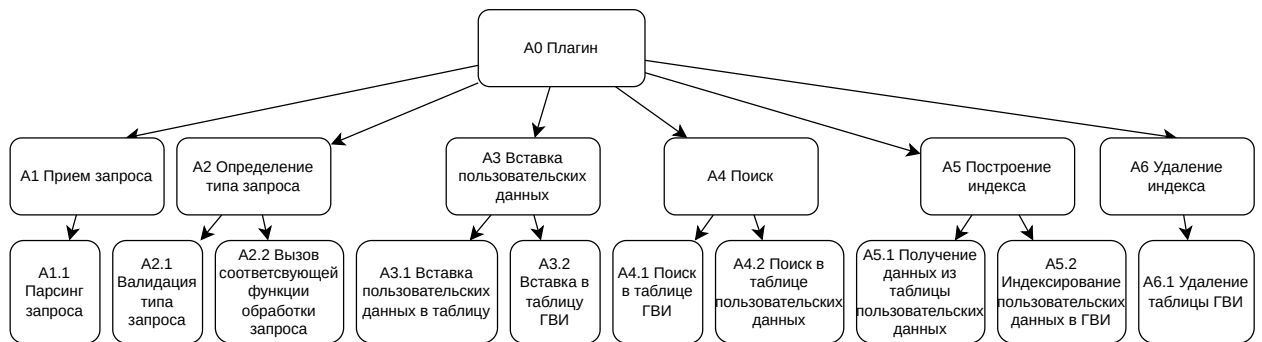


Рисунок 9 — Функциональная декомпозиция плагина

Верхний уровень (A0) — плагин — декомпозируется на шесть функциональных блоков:

- A1 Прием запроса:
  - A1.1 Парсинг запроса.
- A2 Определение типа запроса:
  - A2.1 Валидация типа запроса;
  - A2.2 Вызов соответствующей функции обработки заявок;
- A3 Вставка пользовательских данных:
  - A3.1 Вставка пользовательских данных в таблицу;
  - A3.2 Вставка в таблицу глобального вторичного индекса;
- A4 Поиск:
  - A4.1 Поиск в таблице глобального вторичного индекса;
  - A4.2 Поиск в таблице пользовательских данных;
- A5 Построение индекса:
  - A5.1 Получение данных из таблицы пользовательских данных;
  - A5.2 Индексирование пользовательских данных в глобальном вторичном индексе;
- A6 Удаление индекса:

– А6.1 Удаление таблицы глобального вторичного индекса.

Три уровня представления обеспечивают постепенное раскрытие деталей устройства разрабатываемого плагина: от контекста внедрения до внутренней структуры обработки данных.

## 5 Архитектура плагина

Описание архитектуры плагина определяет структуру системы на уровне компонентов, их обязанности и принципы взаимодействия. Архитектура является ключевым артефактом, обеспечивающим переход от моделей предметной области к реализации. Архитектурное описание выполнено в соответствии с требованиями ГОСТ Р 571000 2016 (ISO/IEC/IEEE 42010:2011) [19].

Плагин расширяет распределенную СУБД Picodata возможностью создания и использования глобальных вторичных индексов. Индекс строится на существующей таблице, автоматически поддерживается в актуальном состоянии и позволяет выполнять эффективный поиск по значениям вторичных ключей на всех узлах кластера.

### 5.1 Структурное представление

На рисунке 10 изображена архитектура плагина. Такое представление позволяет нам логически разбить реализуемую программу на независимые модули.

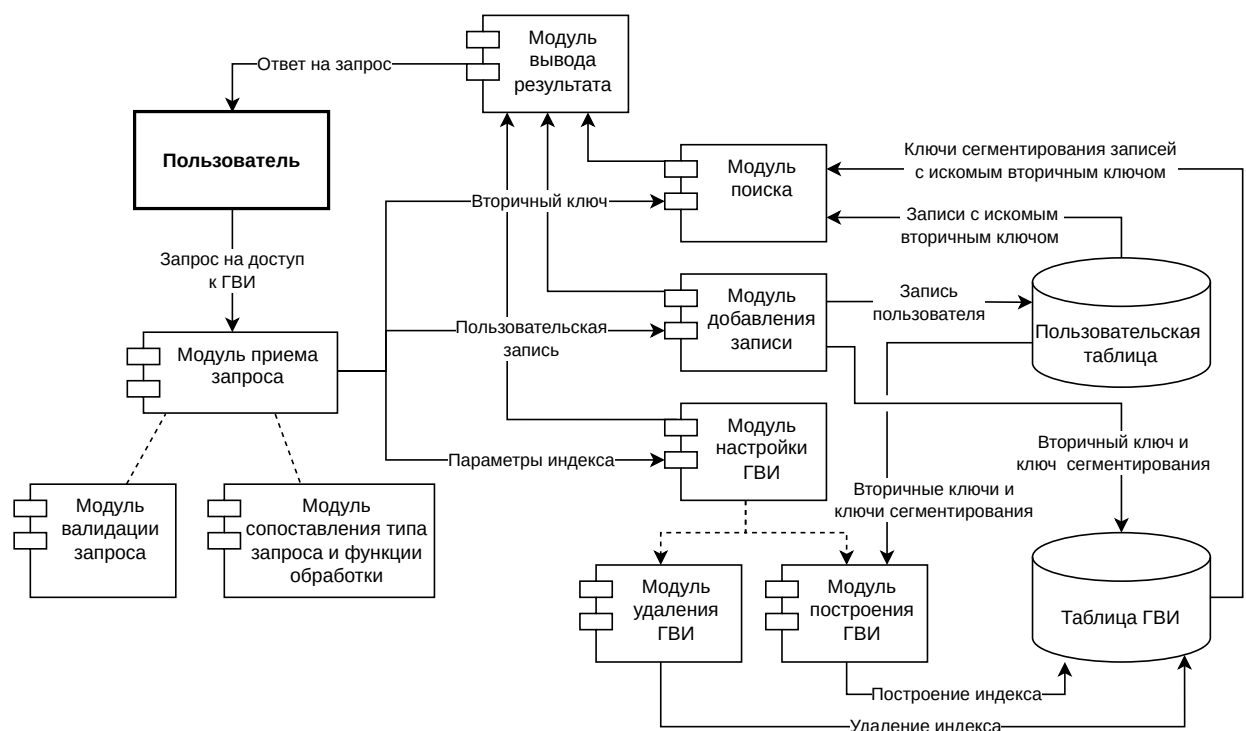


Рисунок 10 — Архитектура плагина ГВИ в СУБД Picodata

На рисунке выделена одна внешняя сущность — пользователь — лицо или сервис, обращающийся к плагину. Остальные сущности: 8 компонент и 2 хранилища являются частями плагина.

Компоненты показаны прямоугольниками с «вилками» с левой стороны, а хранилища — широкими цилиндрами.

Рассмотрим роль каждой компоненты:

- модуль приема запроса — считывает данные, отправленные пользователем, валидирует их, определяет тип запроса, после чего вызывает соответствующую функцию обработки запроса. Он состоит из двух подмодулей:
  - модуль валидации запроса — проверяет, что данные запроса соответствуют некой схеме, никакие инварианты запроса не нарушены, то есть отправлена необходимая информация для обработки;
  - модуль сопоставления типа запроса и функции обработки;
- модуль поиска — реализует механизм поиска данных по вторичному ключу;
- модуль добавления данных — реализует механизм обновления пользовательской таблицы и ГВИ;
- модуль настройки ГВИ — производит операции удаления и добавления глобальных вторичных индексов. Он состоит из двух подмодулей:
  - модуль удаления ГВИ — освобождает место, занятое глобальным вторичным индексом;
  - модуль построения ГВИ — по данным из индексируемой таблицы с нуля строит глобальный вторичный индекс.

Компоненты образуют полный цикл обработки одного запроса пользователя.

На рисунке хранилищами являются таблицы СУБД:

- пользовательская таблица — индексируемая таблица с данными пользователей;
- таблица ГВИ — таблица, представляющая из себя глобальный вторичный индекс.

## 5.2 Диаграммы развертывания

Теперь перейдем к тому как это будет устроено в виде кода.

### 5.2.1 Развертывание плагина на узле

На рисунке 11 представлена диаграмма развертывания плагина в контексте одного узла СУБД Picodata. Диаграмма выполнена в нотации UML, описанной в «Unified Modeling Language Specification, Version 2.5.1» [20] и следует рекомендациям из статьи Creately Blog «Простое руководство к диаграммам развертывания UML» [21].

Выделенным прямоугольником показана внешняя сущность — пользователь. Он направляет запросы напрямую в интерфейс плагина (по HTTP).

Объемными прямоугольниками изображены сущности, являющиеся частью узла Picodata.

Прямоугольниками с пометкой в правом верхнем углу обозначены компоненты плагина.

Прямоугольниками с надписью <<артефакт>> показаны файлы, необходимые для запуска узла с плагином.

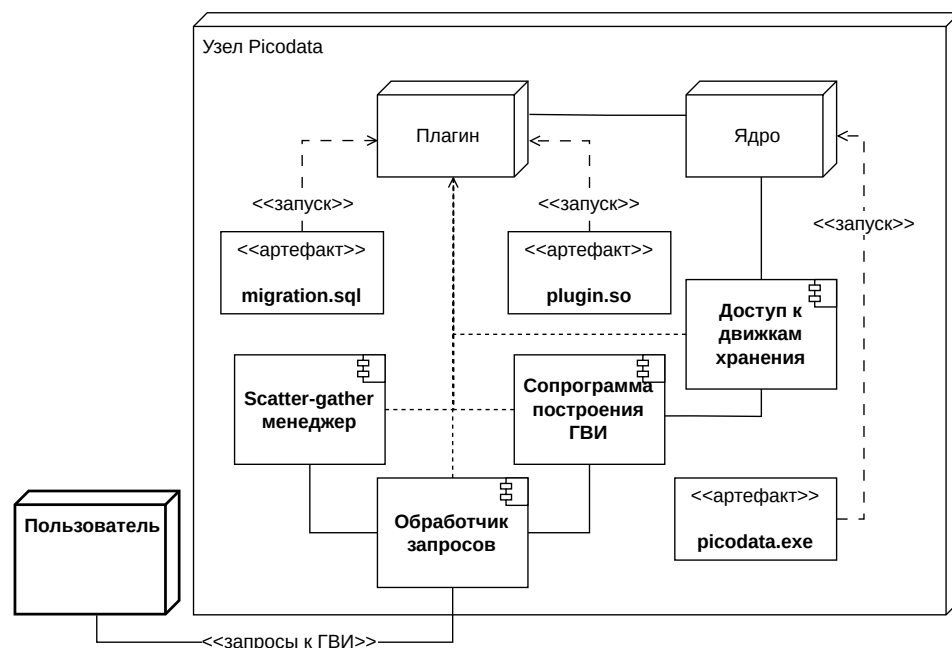


Рисунок 11 — Диаграмма развертывания плагина на одном узле СУБД  
Picodata

Плагин состоит из четырех основных компонентов, реализованных в виде отдельных модулей:

- доступ к движкам хранения:
  - предоставляет функции для работы с хранилищами Picodata
    - чтение и запись в системные таблицы, низкоуровневая работа с таблицами Picodata;
  - выступает интерфейсом хранилища для других компонентов;
- модуль обработки запросов:
  - обрабатывает входящие запросы от пользователей, определяет тип операции и вызывает другие модули;
- модуль построения ГВИ:
  - фоновая сопрограмма, запускаемая при создании индекса, которая обновляет индекс в соответствии с данными пользовательской таблицы;
- модуль scatter-gather:
  - отвечает за алгоритм поиска по индексу. При запросе рассылает подзапросы на все узлы кластера (scatter), собирает и агрегирует результаты (gather), возвращает итоговый набор данных.

На диаграмме изображены следующие артефакты:

- `picodata.exe` — исполняемый файл, реализующий СУБД Picodata;
- `plugin.so` — скомпилированная из исходного кода программа, реализующая взаимодействие с глобальным вторичным индексом;
- `migration.sql` — SQL-команды, объявляющие схему метаданных для использования глобального вторичного индекса.

Прямоугольником с надписью «Ядро» на рисунке 11 обозначен интерфейс, предоставляемый плагину узлом Picodata с помощью библиотеки на языке программирования Rust.

### 5.2.2 Развертывание плагина на кластере

Для полного понимания работы плагина ГВИ в распределенной СУБД Picodata необходимо рассмотреть его развертывание в контексте нескольких



узлов. Так как Picodata является распределенной СУБД, то и плагин тоже децентрализован: его компоненты присутствуют на каждом узле кластера. На рисунке 12 представлена схема разворачивания взаимодействия плагина с пользователем и между узлами.

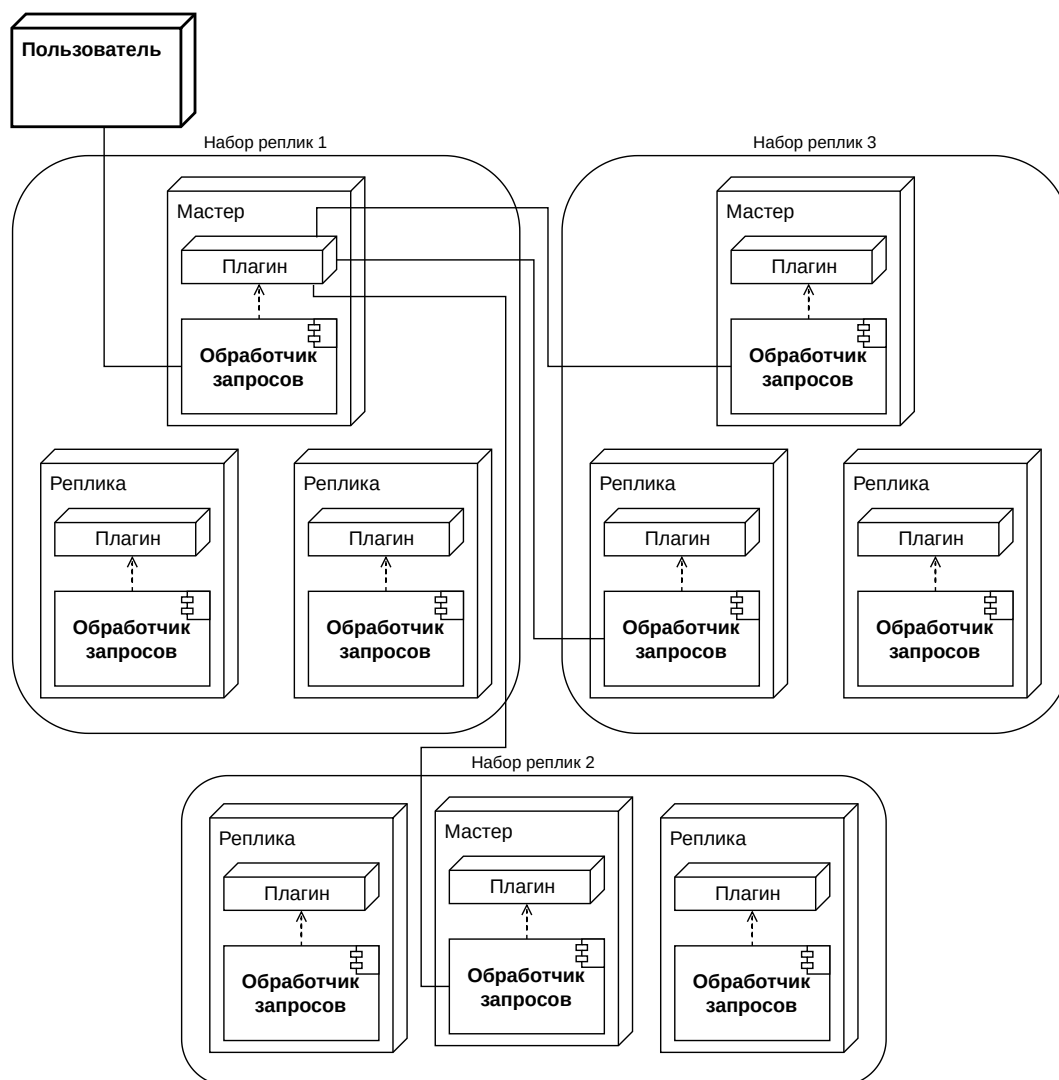


Рисунок 12 — Диаграмма разворачивания плагина в рамках кластера СУБД Picodata

Каждый объемный прямоугольник с подписью «мастер» или «реплика» является уменьшенной версией предыдущей диаграммы. Взаимодействие между узлами осуществляется через внутренние механизмы кластера (RPC-вызовы).

### 5.2.3 Представление данных

Индексные записи хранятся в движке хранения *memtx*, хранящий структуру данных B<sup>+</sup>-дереве в оперативной памяти. Структура служебных

таблицы плагина представлена в таблицах 3 и 4. Угловыми скобками («<» и «>») в названиях таблиц окружены параметры построенного ГВИ:

- table — название индексируемой таблицы;
- field — название индексируемого атрибута.

Таблица 3 — Структура таблицы `gsi_metadata`

| Поле                              | Описание                         |
|-----------------------------------|----------------------------------|
| <code>gsi_index_table_name</code> | Название таблицы ГВИ             |
| <code>indexed_table_name</code>   | Название индексируемой таблицы   |
| <code>indexed_field_name</code>   | Название индексируемого атрибута |

Таблица 4 — Структура таблицы `gsi_<table>_<field>`

| Поле                             | Описание                       |
|----------------------------------|--------------------------------|
| <code>secondary_key_value</code> | Значение вторичного ключа      |
| <code>sharded_key_value</code>   | Значение ключа сегментирования |

Согласованность с исходными данными обеспечивается с гарантией конечной стабильности: обновления применяются асинхронно, но с конечной доставкой.

### 5.3 Итог

Разработанное описание архитектуры ПО представляет собой комплект взаимосвязанных представлений, фиксирующих структурные и поведенческие аспекты плагина ГВИ для СУБД Picodata. Артефакт включает:

- общие сведения и контекст — определено место плагина в экосистеме Picodata, его зависимость от внутренних интерфейсов, внешние сущности;
- архитектурные представления:
  - структурное представление — выделены основные компоненты плагина, описаны их обязанности и взаимодействие между собой;
  - представления развертывания — диаграммы иллюстрирует децентрализованный характер плагина, взаимодействие плагинов между узлами, связь с внешними сущностями и внутреннее устройство плагина;

- представление данных — детализированы структуры служебных таблиц , что задает физическую модель хранения индексной информации. Указан тип хранилища и гарантии согласованности;

Все это повышает прозрачность архитектуры и облегчает ее дальнейшее развитие.

## 6 Описание способов определения показателей качества ПО

Описание показателей качества определяет количественные и качественные характеристики, по которым оценивается соответствие разрабатываемого ПО установленным требованиям. Это необходимо для объективной оценки готовности системы к внедрению и для определения критериев приемки на этапе испытаний (глава 7). Показатели качества выбраны в соответствии с моделью качества ГОСТ Р ИСО/МЭК 25010-2015 [22].

### 6.1 Показатели качества разрабатываемого плагина

Таблица 5 — Метрики качества плагина

| Метрика                                                              | Формула/описание                                                                                                                                                               | Целевое значение |
|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|
| Отношение количеств запросов в секунду при вставке в ГВИ и ЛВИ       | $\frac{RPS_{update}^{GSI}}{RPS_{update}^{LSI}}$                                                                                                                                | $\geq 1.5$       |
| Отношение средних количеств сетевых переходов при поиске в ГВИ и ЛВИ | $\frac{NR_{search}^{GSI}}{NR_{search}^{LSI}}$                                                                                                                                  | $\leq 0.25$      |
| Критическая кардинальность                                           | Отношение кардинальности индексируемого атрибута к кардинальности индексируемой таблицы, при котором среднее время ответа на поисковой запрос с использованием ГВИ и ЛВИ равны | $\geq 0.5$       |
| Доступность СУБД с включенным плагином                               | Процент времени, проведенного СУБД в рабочем состоянии за месяц                                                                                                                | $\geq 99.5\%$    |

Где  $RPS_{update}^{GSI}$  — количество запросов в секунду при вставке в таблицу, индексируемую ГВИ,  $RPS_{update}^{LSI}$  — количество запросов в секунду при вставке в таблицу, индексируемую ЛВИ,  $NR_{search}^{GSI}$  — среднее количество сетевых переходов при поиске через ГВИ,  $NR_{search}^{LSI}$  — среднее количество сетевых переходов при поиске через ЛВИ

## 6.2 Показатели качества ПО по ГОСТ Р ИСО/МЭК 25010

| Характеристика             | Подхарактеристика           | Способ измерения                                                                                                     |
|----------------------------|-----------------------------|----------------------------------------------------------------------------------------------------------------------|
| Функциональная пригодность | Функциональная корректность | Доля расхождений между данными и индексом ( $\leq 0.05$ )                                                            |
| Функциональная пригодность | Функциональная пригодность  | Доля ускоренных запросов ( $\geq 0.9$ )                                                                              |
| Производительность         | Временная эффективность     | Время добавления одной записи в ГВИ ( $\leq 10$ мс)                                                                  |
| Производительность         | Временная эффективность     | Значения перцентилей: 50, 95, 99 поиска одной записи в ГВИ ( $\leq 5$ мс, $\leq 15$ мс, $\leq 50$ мс соответственно) |
| Производительность         | Использование ресурсов      | Трата времени CPU для поддержки ГВИ при разнообразных запросах к СУБД за час ( $< 10\%$ )                            |
| Надежность                 | Доступность                 | Время доступности в месяц ( $\geq 99.5\%$ )                                                                          |
| Надежность                 | Доступность                 | Среднее время между отказами (MTBF $\geq 168$ ч.)                                                                    |
| Защищенность               | Защищенность                | Наличие контроля доступа к функционалу плагина                                                                       |

### **6.3 Способы определения показателей**

Показатели производительности и функциональной пригодности измеряются с помощью нагрузочного тестирования (Locust) и мониторингом метрик с узлов и тестирующего ПО.

Показатели надежности определяются longevity (продолжительным) тестированием СУБД с включенным плагином и непрерывной нагрузкой.

Показатели защищенности определяются путем использования интеграционного тестирования, в которых предпринимается попытка превысить права пользователя.

### **6.4 Характеристика полученного результата**

Разработанный перечень показателей качества представляет собой формализованное описание метрик, по которым будет оцениваться успешность реализации плагина глобальных вторичных индексов. Артефакт включает две группы показателей:

- 1) специфические метрики плагина – количественные характеристики, непосредственно связанные с целями работы. Эти метрики позволяют сравнить эффективность ГВИ с существующими ЛВИ в условиях, определенных в главе 1.
- 2) характеристики качества по ГОСТ Р ИСО/МЭК 25010-2015 – выбраны подхарактеристики, наиболее релевантные для разрабатываемого плагина: функциональная корректность, временная эффективность, использование ресурсов, доступность, защищенность. Для каждой указан способ измерения или целевое значение, что обеспечивает объективность последующей оценки.

Артефакт обладает следующими свойствами:

- полнота – охватывает ключевые аспекты качества: функциональность, производительность, надежность, безопасность;
- измеримость – все показатели имеют количественные или четко определенные качественные критерии, что позволяет провести объективные испытания;

- обоснованность – выбор метрик опирается на анализ проблем текущей реализации и ожидаемые улучшения от внедрения плагина;
- связанность – показатели напрямую используются в главе 7 для разработки программы и методик испытаний.

Использована модель качества ГОСТ Р ИСО/МЭК 25010-2015, что обеспечивает согласованность с общепринятой практикой оценки ПО.

## 7 Программа и методики испытаний ПО

### 7.1 Объект и цели испытаний

Объект испытаний – плагин, реализующий функциональность ГВИ в распределенной СУБД Picodata.

Цели испытаний:

- подтверждение функциональной корректности плагина;
- оценка производительности операций поиска и вставки при использовании ГВИ в сравнении с ЛВИ;
- проверка соответствия показателей качества, установленных в главе 6;
- выявление условий эффективного применения ГВИ.

### 7.2 Состав проверяемых требований

В таблице 7 приведены проверяемые метрики и их целевые значения (согласно разделу 6.2).

Таблица 7 — Проверяемые метрики качества плагина

| Метрика                                                    | Обозначение         | Целевое значение | Раздел ГОСТ                 |
|------------------------------------------------------------|---------------------|------------------|-----------------------------|
| Отношение запросов вставки в секунду (ГВИ / ЛВИ)           | $R_{\text{insert}}$ | $\geq 1.5$       | Производительность          |
| Отношение средних сетевых переходов при поиске (ГВИ / ЛВИ) | $R_{\text{net}}$    | $\leq 0.25$      | Временная эффективность     |
| Критическая кардинальность                                 | $C_{\text{crit}}$   | $\geq 0.5$       | Временная эффективность     |
| Доступность СУБД с плагином                                | $A$                 | $\geq 99.5\%$    | Надежность                  |
| Доля расхождений данных и индекса                          | $E_{\text{corr}}$   | $\leq 0.05$      | Функциональная корректность |



| Метрика                                      | Обозначение         | Целевое значение  | Раздел ГОСТ             |
|----------------------------------------------|---------------------|-------------------|-------------------------|
| Время добавления одной записи в ГВИ          | $T_{\text{insert}}$ | $\leq 10$ мс      | Временная эффективность |
| Перцентили времени поиска (50/95/99)         | $T_{\text{search}}$ | $\leq 5/15/50$ мс | Временная эффективность |
| Дополнительная загрузка CPU на поддержку ГВИ | $U_{\text{cpu}}$    | $< 10\%$          | Использование ресурсов  |
| Среднее время между отказами (MTBF)          | MTBF                | $\geq 168$ ч      | Надежность              |
| Контроль доступа к функциям плагина          |                     | Наличие           | Защищенность            |

### 7.3 Методы испытаний

Для каждой группы метрик применяются специализированные методики.

#### 7.3.1 Функциональная корректность

Проверяется путем выполнения эталонных сценариев:

- создание ГВИ на существующей таблице;
- вставка, обновление, удаление записей в индексируемой таблице;
- выполнение поисковых запросов с использованием ГВИ;
- сравнение результатов запросов с данными, полученными прямым сканированием таблицы (без индекса) или через ЛВИ.

Для каждого действия фиксируется количество расхождений. Допустимая доля расхождений  $E_{\text{corr}} \leq 0.05$  (не более 5% от общего числа проверенных записей).

### 7.3.2 Производительность операций вставки

Используется нагрузочное тестирование с помощью инструмента Locust. Сценарий:

- 1) разворачивается кластер Picodata варьирующей топологией (1 набор реплик с 1 узлом, 3 набора реплик по 3 узла, 10 наборов реплик по 3 узла, 25 наборов реплик по 5 узлов);
- 2) создается тестовая таблица с ЛВИ;
- 3) выполняется вставка записей с интенсивностью, обеспечивающей утилизацию CPU не более 80%. Измеряется количество успешных операций в секунду  $RPS_{update}^{LSI}$ ;
- 4) удаляется тестовая таблица с ЛВИ;
- 5) создается такая же таблица с ГВИ;
- 6) выполняется вставка записей с интенсивностью, обеспечивающей утилизацию CPU не более 80%. Измеряется количество успешных операций в секунду  $RPS_{update}^{GSI}$ ;
- 7) эксперимент повторяется не менее 5 раз, вычисляется среднее значение  $R_{insert} = \frac{RPS_{update}^{GSI}}{RPS_{update}^{LSI}}$ . Требование  $R_{insert} \geq 1.5$  по крайней мере при одной из топологии. Это послужит для составления списка рекомендаций конфигурации кластера по использованию ГВИ.

### 7.3.3 Сетевые переходы при поиске

Для измерения числа сетевых переходов в ГВИ используется встроенный мониторинг Picodata (статистика RPC-вызовов). Сценарий:

- 1) таблица заполняется данными с параметризированной кардинальностью индексируемого поля;
- 2) выполняется серия из 10000 поисковых запросов по случайным значениям;
- 3) для каждого запроса подсчитывается количество сетевых запросов – при использовании ЛВИ – это значение вычисляется по формуле, при ГВИ – прямым подсчетом статистики из мониторинга;
- 4) вычисляется среднее количество сетевых переходов для ГВИ и для ЛВИ, затем отношение  $R_{net} = \frac{NR_{search}^{GSI}}{NR_{search}^{LSI}}$ . Требование  $R_{net} \leq 0.25$ .

### 7.3.4 Критическая кардинальность

Проводится серия экспериментов с различной кардинальностью индексируемого поля (от 0.1 до 1.0 с шагом 0.1). Для каждого значения:

- измеряется среднее время ответа для поискового запроса через ЛВИ и через ГВИ (по 10000 запросов);
- строится график зависимости времени от кардинальности.
- определяется точка пересечения кривых, которая дает  $C_{crit}$ .  
Требование  $C_{crit} \geq 0.5$ .

### 7.3.5 Доступность и надежность

Проводится longevity-тестирование: кластер с включенным плагином непрерывно работает под смешанной нагрузкой (чтение/запись) в течение 7 суток (168 часов). Фиксируются:

- время простоев (если произошел отказ);
- количество отказов;
- MTBF (среднее время между отказами) – вычисляется как отношение общего времени наблюдения к числу отказов (при отсутствии отказов считается бесконечным, но для оценки достаточно, что  $MTBF \geq 168$  ч);
- доступность  $A = \frac{\text{общее время} - \text{время простоев}}{\text{общее время}} \cdot 100\%$ . Требование  $A \geq 99.5\%$ .

### 7.3.6 Загрузка CPU

Во время выполнения longevity-теста и нагрузочных тестов собирается статистика использования CPU на каждом узле (средняя, пиковая). Сравнивается загрузка при выключенном и включенном плагине (при одинаковой нагрузке). Дополнительная трата времени CPU  $\Delta U$  не должна превышать 10% от базовой загрузки без плагина.

### 7.3.7 Контроль доступа

Проверяется, что операции создания/удаления индекса, а также прямые запросы к служебным таблицам доступны только пользователям

с соответствующими привилегиями. Тестирование включает попытки выполнения команд от имени пользователя без прав – они должны завершаться отказом с кодом ошибки авторизации.

#### **7.4 Средства испытаний**

- аппаратные средства: кластер из 3–5 виртуальных машин с одинаковой конфигурацией (CPU, RAM, диск). Каждая машина выступает в нескольких узлов Picodata (это рекомендуемая разработчиками конфигурация);
- программные средства:
  - СУБД Picodata;
  - плагин ГВИ;
  - инструменты нагрузочного тестирования: Locust с использованием драйвера Picodata;
  - инструменты мониторинга: встроенные статистики Picodata, Prometheus, Grafana;
  - средства автоматизации: Ansible для развертывания кластера, Jupyter Notebook для обработки результатов.

#### **7.5 Условия и порядок проведения испытаний**

Условия:

- кластер должен быть изолирован от внешней нагрузки;
- перед каждым экспериментом кластер перезапускается, данные загружаются заново (для обеспечения повторяемости);
- параметры нагрузки (топология кластера, количество запросов, кардинальность и другие) фиксируются в скриптах и сохраняются в журнале эксперимента.

Порядок проведения:

- 1) подготовка кластера (запуск Picodata, загрузка плагина);
- 2) функциональное тестирование (проверка корректности);
- 3) нагрузочное тестирование производительности (вставка, поиск);
- 4) longevity-тест (7 суток);

- 5) обработка результатов, вычисление метрик, сравнение с целевыми значениями.

## 7.6 Критерии приемки

Плагин считается успешно прошедшим испытания, если все проверяемые метрики из таблицы 7 удовлетворяют целевым значениям. Допускаются незначительные отклонения (в пределах погрешности измерений), но они должны быть задокументированы и обоснованы.

## 7.7 Сравнение полученных результатов с математическими моделями

На рисунке 13 представлен график плотности вероятности времени ожидания ответа от СУБД при поиске в ЛВИ при следующих входных параметрах модели:

```
intensity_search_LSI: 10000
replicaset_count: 15
cluster_time_k: 1.5
cluster_time_theta: 0.02
cluster_net_speed: 4096
user_time_k: 1.5
user_time_theta: 0.1
user_net_speed: 1000000
row_size: 128
data_cardinality: 1000
fk_cardinality: 100
cpu_frequency: 10000000
mem_frequency: 10000000
wal_time: 0.002
timeout: 2
btree_order: 3
```

Листинг 1 — Параметры тестовой математической модели

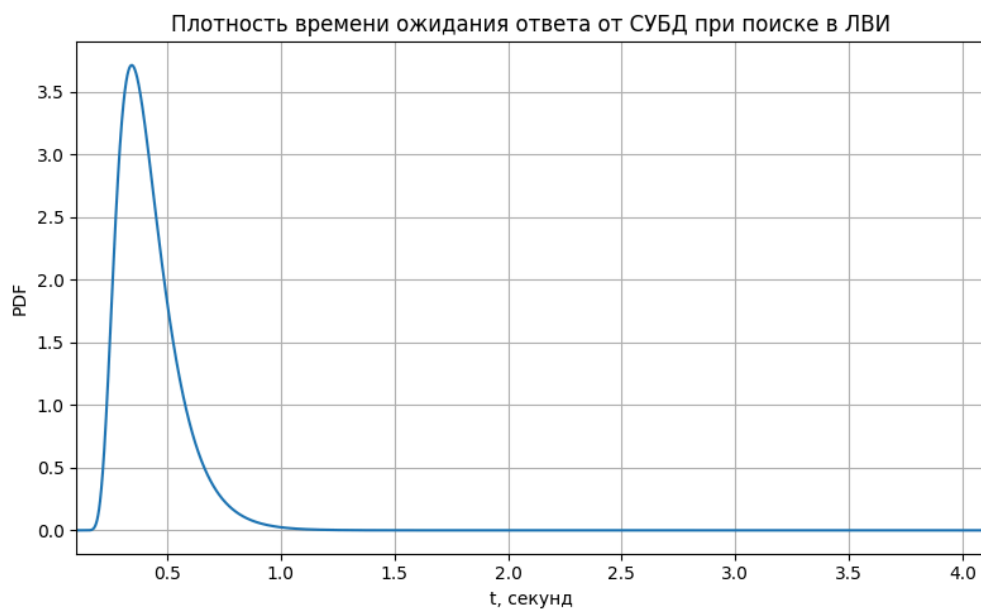


Рисунок 13 — Плотность вероятности времени ожидания ответа от СУБД при поиске в ЛВИ

Показатели эффективности модели с данными параметрами равны:

$$\lambda_{\text{search,LSI}}^{\text{sub,max}} \approx 714285 \quad (128)$$

$$M\left[T_{\text{search,LSI}}^{\text{user}}\right] \approx 0.42 \quad (129)$$

$$W_{\text{search,LSI}}^{\text{total,q}} \approx 0.000001 \quad (130)$$

$$P\left(T_{\text{search,LSI}}^{\text{user}} \geq T_{\text{timeout}}\right) \approx 0.0000004 \quad (131)$$

## ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы решена задача повышения эффективности поиска по вторичным ключам в распределенной СУБД Picodata путем разработки и исследования глобального вторичного индекса (ГВИ). Получены следующие результаты:

- 1) на основе анализа архитектуры Picodata и теории массового обслуживания построена математическая модель функционирования локального (ЛВИ) и глобального (ГВИ) вторичных индексов. Модель в единой формальной системе описывает оба типа индексов для операций поиска и обновления, позволяя вычислять максимальную интенсивность входного потока, среднее время отклика, суммарное время ожидания в очередях и вероятность тайм-аута;
- 2) получены аналитические выражения для показателей эффективности. Показано, что пропускная способность поиска в ГВИ не уступает ЛВИ даже при наихудшем распределении ключей, а при высокой кардинальности вторичного ключа может многократно превосходить ее за счет сокращения числа сетевых взаимодействий. В то же время обновление в ГВИ требует примерно вдвое больших затрат из-за двухэтапной фиксации изменений;
- 3) разработан программный плагин, реализующий функциональность ГВИ в СУБД Picodata: создание, автоматическое обновление, удаление индекса, а также распределенный поиск с его использованием. Плагин децентрализован, работает на каждом узле кластера и использует штатные механизмы хранения Picodata;
- 4) определены ключевые показатели качества плагина в соответствии с ГОСТ Р ИСО/МЭК 25010-2015, разработана программа и методики испытаний, включающие нагрузочное, longevity-тестирование и проверку функциональной корректности;
- 5) на основе теоретического анализа сформулированы условия предпочтительного применения ГВИ: высокая кардинальность индексируемого атрибута, большое число наборов реплик, преобладание поисковых запросов над операциями обновления. При низкой кардинальности или интенсивном потоке обновлений более эффективным остается ЛВИ.

Таким образом, поставленная цель достигнута, все задачи решены. Разработанная модель, программное обеспечение и рекомендации могут служить основой для дальнейшего совершенствования механизмов индексирования в распределенной СУБД Picodata.



## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Википедия. В+-дерево [Электронный ресурс]. 2026. URL: <https://ru.wikipedia.org/wiki/B%E2%81%BA-%D0%B4%D0%B5%D1%80%D0%B5%D0%B2%D0%BE>.
2. Википедия. В\*-дерево [Электронный ресурс]. 2026. URL: [https://ru.wikipedia.org/wiki/B\\*-%D0%B4%D0%B5%D1%80%D0%B5%D0%B2%D0%BE](https://ru.wikipedia.org/wiki/B*-%D0%B4%D0%B5%D1%80%D0%B5%D0%B2%D0%BE).
3. Tarantool Developers. Tarantool source code: bps\_tree.h [Электронный ресурс]. 2025. URL: [https://github.com/tarantool/tarantool/blob/4e27591f685e9c695853a0165d4081148d12e315/src/lib/salad/bps\\_tree.h](https://github.com/tarantool/tarantool/blob/4e27591f685e9c695853a0165d4081148d12e315/src/lib/salad/bps_tree.h) (дата обращения: 11.04.2026).
4. Бабин О. Как написать свой индекс в Tarantool [Электронный ресурс]. 2020. URL: <https://habr.com/ru/companies/vk/articles/505880/>.
5. Портал документации Picodata [Электронный ресурс]. ООО "Пикодата", 2026. URL: <https://docs.picodata.io/picodata/stable/> (дата обращения: 02.03.2026).
6. Kleppmann M. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. Sebastopol, CA: O'Reilly Media, 2017.
7. Chen P. P.-S. The entity-relationship model—toward a unified view of data // ACM Transactions on Database Systems. Association for Computing Machinery (ACM), 1976. Т. 1, № 1. Сс. 9–36.
8. Вентцель Е. С. Исследование операций: задачи, принципы, методология. 2-е изд. "Наука" Главная редакция физико-математической литературы, 1988.
9. Розенберг В. Я., Прохоров А. И. Что такое теория массового обслуживания. 2-е изд. Москва: Советское радио, 1965. С. 256.
10. Shortle J. F. и др. Fundamentals of Queueing Theory. 5-е изд. Hoboken, NJ: Wiley, 2018.
11. Liu Y., Li J., Gu N. Modeling Internet Link Delay Based on Measurement // 2009 International Conference on Computational Science and Engineering. Vancouver, BC, Canada: IEEE, 2009. Т. 4. Сс. 874–879.

12. Valadão E. D. и др. Caracterização de tempos de ida-e-volta na Internet // Revista de Informática Teórica e Aplicada. Porto Alegre, Brazil: UFRGS, 2012. Т. 19, № 2. Сс. 4–20.
13. France R. и др. The UML as a formal modeling notation // Computer Standards & Interfaces. Elsevier BV, 1998. Т. 19, № 7. Сс. 325–334.
14. Кибзун А. И., Горяинова Е. Р., Наумов А. В. Теория вероятностей и математическая статистика. Базовый курс с примерами и задачами. 3-е изд. Москва: Физматлит, 2011.
15. Ross S. M. A First Course in Probability. 10-е изд. Pearson, 2019.
16. Leung C. H. C. Approximate storage utilisation of B-trees: A simple derivation and generalisations // Information Processing Letters. Elsevier, 1984. Т. 19, № 4. Сс. 199–201.
17. System context diagram [Электронный ресурс]. Wikimedia Foundation, 2025. URL: [https://en.wikipedia.org/wiki/System\\_context\\_diagram](https://en.wikipedia.org/wiki/System_context_diagram) (дата обращения: 01.03.2026).
18. Бесков Д. Контекстная диаграмма [Электронный ресурс]. URL: <https://systems.education/context-diagram#showmore> (дата обращения: 02.03.2026).
19. ГОСТ Р 57100–2016. Системная и программная инженерия. Описание архитектуры. Москва, 2016.
20. Unified Modeling Language Specification, Version 2.5.1: technical report. 2017.
21. Creately Blog. Простое руководство к диаграммам развертывания UML [Электронный ресурс]. 2022.
22. ГОСТ Р ИСО/МЭК 25010–2015. Системная и программная инженерия. Требования и оценка качества систем и программных средств (SQuaRE). Модели качества систем и программных продуктов. Москва, 2015.